

This talk is a practitioner's tour of the Catapult HLS user-view scheduling rules and the formal architecture behind them.

The audience is hardware architects and design verification engineers. You do not need to be an HLS expert or a formal-methods specialist. The goal is that you leave knowing: what the contract lets HLS do to your design, what it forbids, how to write source models and testbenches that stay inside it, and what the formal guarantees actually say - and do not say.

Theorem and appendix names appear in small print so you can find the precise statement in the rules document, but the talk keeps the formal vocabulary to a minimum.

EXECUTIVE SUMMARY

A small contract that lets verification stay at the source

HLS may add latency, pipeline loops and reorder memory accesses. The scheduling contract states exactly what it may not change — so a testbench written for the SystemC model stays meaningful for the RTL.



The goal

Predictable source-to-RTL behavior: verify and debug mostly at the source, with one testbench for both models.



The rules

Three classes of IO — message passing, signal IO and synchronization — each with one uniform rule.



The freedom

Pipelining, added states, late sampling of stable inputs, permitted memory reordering, finite buffering.



The guarantees

With the required evidence: proved source interface properties hold on RTL, and the required workload completes. Lazy pre-HLS checks preferred.



Your part

Latency-insensitive interfaces, well-anchored signal IO, explicit memory, reset and channel structure.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

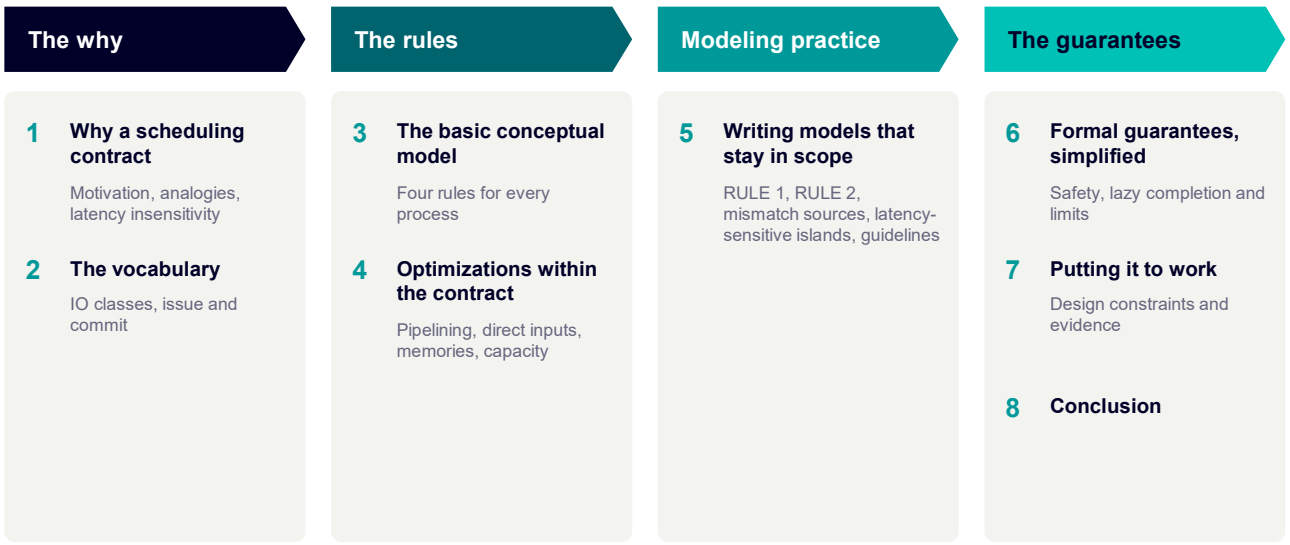
Five ideas carry the whole talk.

1. The goal is predictable source-to-RTL behavior: most verification and debug happen on the pre-HLS SystemC model, and one testbench serves both the model and the RTL.
2. The rules are simple. There are three classes of IO operation - message passing, signal IO and synchronization - and each is scheduled by one uniform rule.
3. Simplicity does not cost QOR: HLS keeps the freedom to pipeline, add states, sample stable inputs late, reorder memory accesses that are proven conflict-free, and implement finite buffering.
4. Two formal results back the rules: a safety result (covered RTL behavior is explained by the reference model) and a completion result (required outputs eventually arrive).
5. The architect's part is to choose interfaces and structures that make that evidence possible.

Where eligible, completion can be checked with a lazy pre-HLS simulation and no per-stimulus RTL run. Section 6 gives the conditions and the evidence that is still required.

AGENDA

Agenda



Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Four acts.

The why: what problem the contract solves, and the handful of terms you need.

The rules: the basic conceptual model, then the optimizations HLS may still perform inside it.

Modeling practice: how to write source models and testbenches that stay inside the contract.

The guarantees: what the formal results say, how they are built from local evidence, what they do not claim, and who supplies which evidence.

Section 6 now introduces the lazy completion workflow, its source-only demand mechanism, a staging example, and its relationship to the standard floor route.

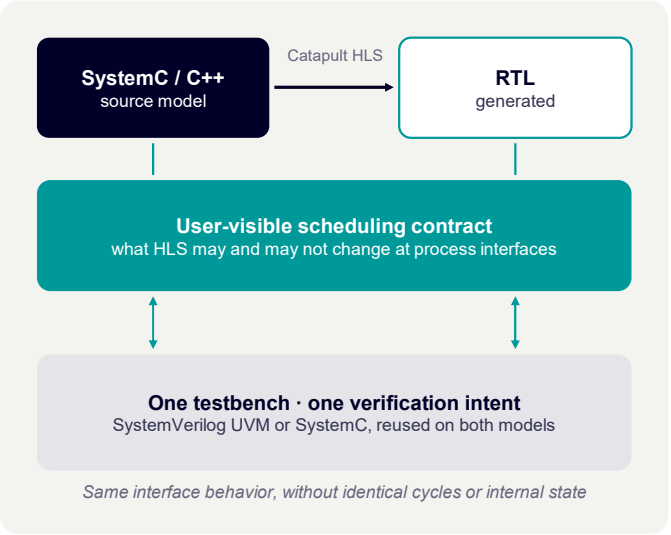
HLS pays off only if verification stays at the source

The problem

- ✕ RTL verification is among the most costly and time-consuming parts of a hardware flow.
- ✕ Generated RTL can differ substantially from the source model in structure and in timing.
- ✕ If teams must return to RTL as the primary level for verification and debug, HLS loses much of its advantage.

The aim

Designers and DV engineers reason primarily at the source level, while keeping a precise way to qualify the resulting implementation.



Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Start with the economics. RTL verification is one of the most expensive and slowest parts of a hardware flow, and machine-generated RTL can look very different from the source in structure and in timing. If teams must return to that RTL as the primary level for functional verification and debug, much of the advantage of HLS is lost.

The scheduling contract is the bridge. It is user-visible and tool-neutral: it states what HLS may and may not change at process interfaces. That lets the same verification intent - for example a UVM or SystemC testbench - be reused across the source reference model and the RTL, without requiring the RTL to preserve every source cycle or internal state.

The testbench author should be able to rely on the interface contract without knowing the HLS tool’s internal scheduling.

An analogy from RTL synthesis

In Verilog

```
Out1 <= #some_delay new_val;
```

- Simulation honors the delay. Longer than a clock period, it changes cycle-level behavior.
- RTL synthesis ignores every delay — and may warn: “simulation and synthesis results are likely to mismatch.”
- Nobody expects synthesis to match the simulation exactly. Everyone aligns on the conceptual model.

	RTL synthesis	HLS with this contract
Conceptual model	Clocked RTL semantics	The user-view scheduling rules
Simulation model	Verilog simulation, including #delays	SystemC / C++ simulation of the source
Synthesis	Ignores delays; flags suspicious ones	Schedules within the rules; flags RULE 1 / RULE 2 violations

Why it works: the conceptual model is simple enough for users and tool vendors to align around, so small gaps between simulation and synthesis are easy to manage.

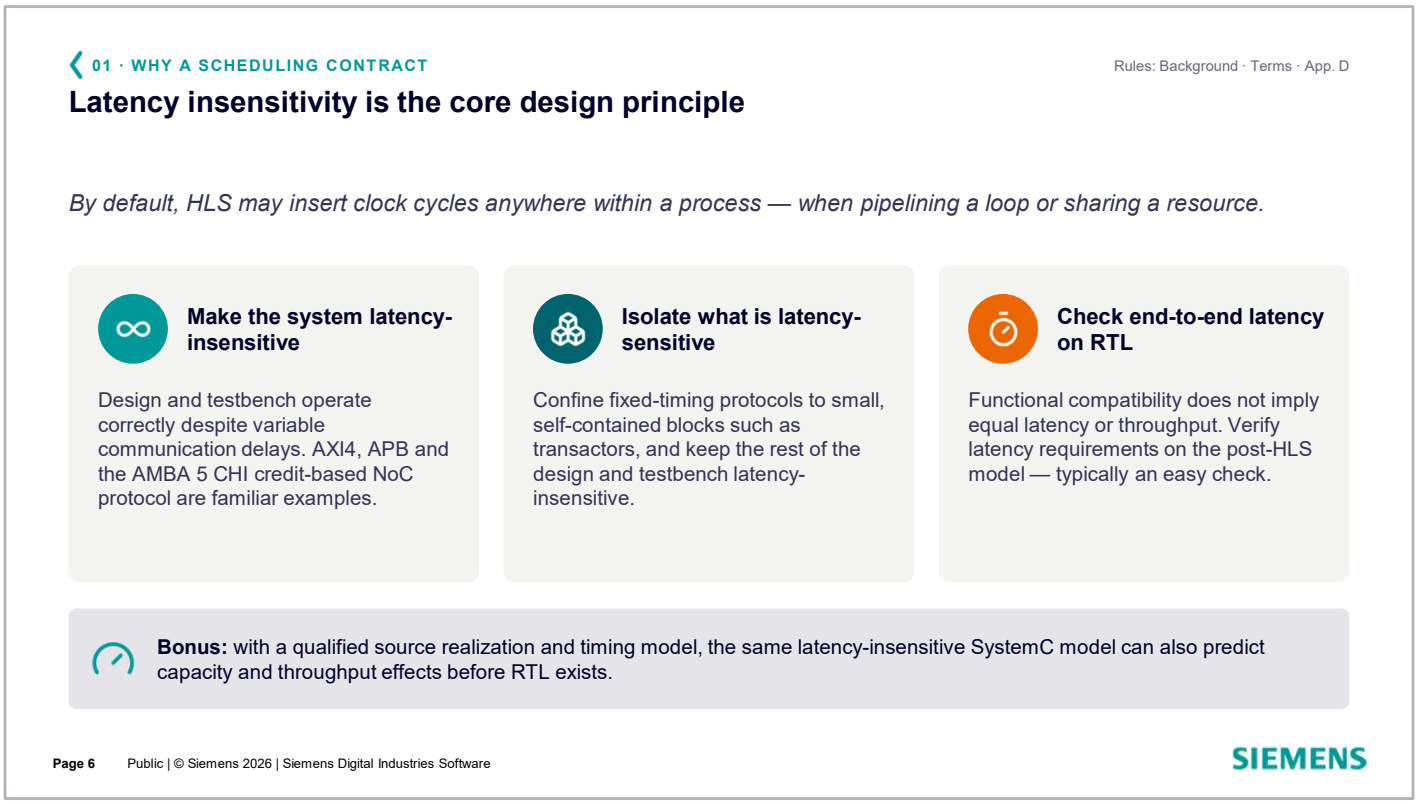
Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The rules document opens with a familiar analogy. In Verilog, 'Out1 <= #some_delay new_val' simulates with the delay. If the delay is shorter than the clock period it probably does not change cycle behavior; if it is longer, it probably does. RTL synthesis ignores all delays anyway, and good tools warn that simulation and synthesis are likely to mismatch, or reject the model.

Nobody argues that RTL synthesis should match Verilog simulation exactly. The flow works because RTL is a simple conceptual model that users and tool vendors align around, and the small gaps between the simulation model and the conceptual model are easy to manage.

The scheduling rules play the same role for HLS. And the RULE 1 / RULE 2 diagnostics we will see later are the HLS counterpart of that 'delay greater than clock period' warning.



By default HLS may add clock cycles anywhere inside a process - to pipeline a loop or to share a multiplier. So the overall approach is to make the whole design and testbench latency-insensitive to the maximum extent possible, while still enabling the key hardware optimizations.

Latency-insensitive means the system works correctly despite variable communication delays. AXI4, APB and AMBA 5 CHI are familiar examples.

Where a protocol really is latency-sensitive, isolate it in a small self-contained block and keep the rest latency-insensitive.

End-to-end latency requirements still matter. The recommendation is to keep the latency-insensitive modeling style and check latency on the post-HLS model, which is usually easy. Functional trace compatibility alone does not establish latency or throughput equality - those need their own qualified timing model.

A factory analogy for the rules

People at workbenches	»	**Processes**
Their written instructions	»	**C++ code in a process**
Objects handed between people	»	**Messages (blocking or non-blocking)**
A wall clock everyone can see	»	**The hardware clock**
Flags, seen at the next tick	»	**Signals (delayed update)**
Subordinates doing subtasks	»	**Pipeline stages**
Meeting at a barrier	»	**Explicit synchronization**
Raising a reset flag	»	**Hardware reset**

Now assume

- Task and transfer times vary
- Everyone works as quickly as they can
- Several products — chairs in different colors and styles — are built at once

To produce furniture reliably, everyone must follow rules like those in this contract.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Appendix A grounds the rules in a factory that builds wooden furniture.

People at workbenches are processes; their written instructions are the C++ code. They pass objects - messages - to each other, blocking or non-blocking. A wall clock that everyone can see is the hardware clock. Colored flags are signals, and a flag change is seen by others only at the next tick of the second hand: that is the delayed-update semantics of signals between sequential processes. Hiring subordinates to do subtasks is pipelining. Agreeing to meet at a barrier is explicit synchronization, and a reset flag restarts work.

Now add realistic assumptions: task and transfer times vary, everyone works as fast as they can, and the factory makes several products at once. To produce furniture reliably, everyone has to follow rules like the ones in this contract. The rules are not arbitrary; they are what predictable concurrent work needs.

Rules: Appendix A

In this example, overlap works; reversal can deadlock

```
// Person 1
seats.Push(s);
backs.Push(b);
```

```
// Person 2
s = seats.Pop();
b = backs.Pop();
```

Blocking calls, zero storage between people: a Push completes only when the matching Pop does.

Each transfer takes at least one minute.

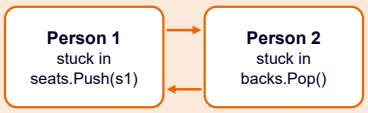
As written: 5 minutes

#	Person 1	Person 2
1	seats.Push(s1)	
2	backs.Push(b1)	s1 = seats.Pop()
3	seats.Push(s2)	b1 = backs.Pop()
4	backs.Push(b2)	s2 = seats.Pop()
5		b2 = backs.Pop()

Both overlap IO: 3 minutes

#	Person 1	Person 2
1	seats.Push(s1) backs.Push(b1)	
2	seats.Push(s2) backs.Push(b2)	s1 = seats.Pop() b1 = backs.Pop()
3		s2 = seats.Pop() b2 = backs.Pop()

Person 2 reverses: deadlock



Person 1 waits for a seats.Pop that never comes; Person 2 waits for a backs.Push that never starts.

Default rule: message calls may overlap, but their source issue order must be preserved. Completion needs separate evidence.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

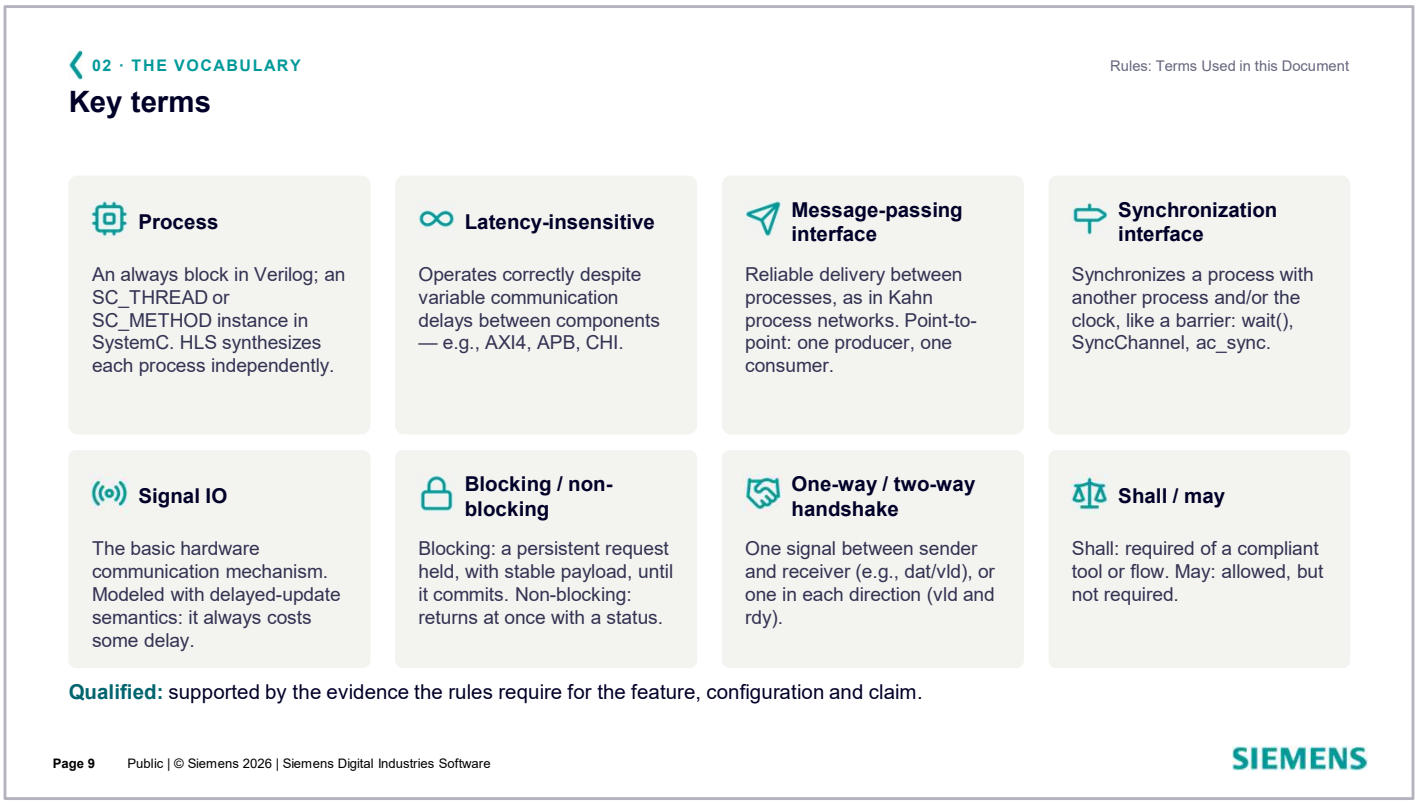
Walk through the tables in order. With the calls as written, Person 1 offers the first seat, then its back, then the second seat and back. The two workers' handoffs occupy the five numbered minutes shown. Each transfer takes at least one minute, and there is no storage between the workers.

When both workers overlap their independent seat and back calls while preserving issue order, the same work finishes in the three minutes shown. A later request may begin before an earlier one finishes.

Now reverse Person 2's calls. Person 1 waits to hand over a seat while Person 2 waits for a back. Neither reaches the matching call, so the first handoff cannot complete.

This example establishes a legal overlap and a specific deadlock caused by reversing issue order. It does not establish that every overlap is harmless. Finite storage, early requests, controllers and the environment still need their own safety and completion evidence.

(References: rules Appendix A, E3 and Appendix K.)



A few terms recur throughout.

A process is an always block in Verilog or an SC_THREAD / SC_METHOD instance in SystemC. HLS finds the processes and synthesizes each one independently.

Message-passing interfaces are the Kahn-process-network style of communication: reliable, ordered delivery. In this document every channel is point-to-point, with exactly one producer and one consumer. Shared resources are modeled with explicit arbiter processes.

Synchronization interfaces are like barriers - wait(), SyncChannel, ac_sync.

Signals always cost some delay; HDLs and SystemC model this with delayed-update semantics.

Blocking means a persistent request held until the operation commits; non-blocking returns immediately with a status. 'Shall' is required of a compliant tool or flow, 'may' is permitted.

Qualified means supported by the evidence the rules require for the feature, configuration and claim. It describes a specific scope of evidence, not a general certification of a tool or every design it can produce.

Three classes of IO operation

Message passing

ac_channel methods; Push, Pop, PushNB, PopNB (e.g., Connections In<> / Out<>)

Moves data between processes. May shift and overlap in time.

Synchronization

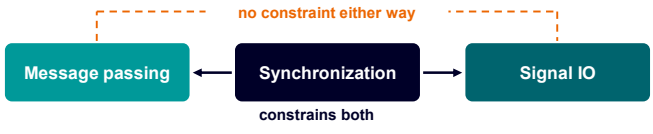
wait(), ac_wait, SyncChannel (sync_in / sync_out), ac_sync

Defines the boundaries the other operations respect. Always stays in source order.

Signal IO

sc_signal / sc_in / sc_out reads and writes; C++ direct inputs

Sequential IO: anchored to synchronization. Combinational IO: observed after settling.



Message calls never move a signal's anchor, and signal statements add no message-order constraints. No "protocol regions" are needed.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Every scheduling rule talks about just three classes of IO operation.

Message passing: all ac_channel methods and the SystemC Push, Pop, PushNB and PopNB calls - for example Matchlib Connections In and Out ports.

Synchronization: SystemC wait and ac_wait, plus SyncChannel and ac_sync.

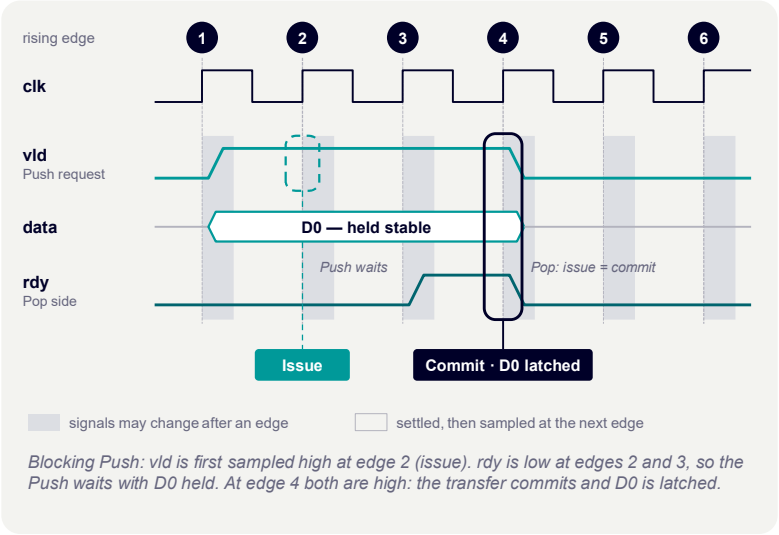
Signal IO: SystemC signal and port reads and writes, plus C++ model direct inputs.

The relationship between them is the key to the whole contract. Synchronization calls constrain both other classes: they anchor signal IO and fence message passing. But message passing and signal IO do not constrain each other's scheduling - a Pop placed between a wait and a signal read does not move that read. Signal values can of course still determine payloads and control flow.

These are the same operations verification engineers already use in testbenches, which is why the rules are easy to adopt. Note what is absent: no 'protocol regions' or similar tool-specific constructs.

Sequential-process signal accesses use synchronization anchors. For combinational processes, observe the stable values after the network settles, under the applicable combinational rules (R2C). Ordinary latency-insensitive interfaces do not themselves establish the selected proof profile.

Issue and commit: the two moments of a transfer



Blocking Push: vld is first sampled high at edge 2 (issue). rdy is low at edges 2 and 3, so the Push waits with D0 held. At edge 4 both are high: the transfer commits and D0 is latched.

Issue

The first clock edge that samples the process's request high: valid for a Push, ready for a Pop.

Commit

The edge that samples valid and ready both high. The data is latched there, and the transfer is mandatory.

Persistent requests

A blocking request stays asserted, payload stable, until commit or reset — no try-and-withdraw.

Registered results

A result returned at edge t can drive a dependent request no earlier than edge t + 1.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Reading the diagram: each numbered cycle is identified with the rising edge that ends it, which Core.TransferLaw calls sampling edge t. Cycle t runs from just after edge t − 1 to edge t, and what the process requests during cycle t is sampled at edge t. Registered outputs change just after an edge (grey bands) and are settled before the next edge. The bands illustrate the convention, not a timing specification: combinational paths may settle later, provided they are settled at the sampling edge. “At the start of cycle t” therefore means the state just after edge t − 1.

Issue is the first edge at which the process’s request is sampled high at its channel boundary: valid for a Push, ready for a Pop. Here vld rises after edge 1, so the Push issues at edge 2, not where the vld waveform starts to rise. Commit is the edge at which valid and ready are both sampled high (edge 4 here), and D0 is latched there. Once settled valid and ready are both high at an eligible edge, the transfer must occur with the data sampled at that edge (Core.TransferLaw).

rdy is first sampled high at edge 4. If the ready side is a blocking Pop by a process, its issue and commit therefore fall on the same edge: issue ≤ commit, with equality allowed (Appendix C). If the ready side is a FIFO or other non-process endpoint, rdy simply reflects space.

Any grant, guard or resource restriction must already be reflected in the settled handshake signals and supporting state. It cannot cancel a transfer after both signals have been sampled high.

A blocking request persists until commit or an affecting reset. A Push also holds its payload stable. There is no try-and-withdraw behavior.

For sequential processes, a result returned at edge t cannot generate a dependent request sampled at that same edge. The earliest dependent request is first sampled at edge t + 1 (Core.RegSeq).

Requests on different interfaces may overlap while preserving source issue order. That does not impose a strict completion order between those interfaces. Also distinguish a physical movement through internal staging from a source-level issue or completion: the declared interface mapping must say which physical activity represents each logical event.

(References: Issue vs. Commit, Appendix C, Core.IC, Core.TransferLaw and Core.RegSeq.)

The basic conceptual model in four rules

1

Synchronization calls stay in source order.

wait(), SyncChannel and ac_sync are never reordered within a process.

2

Signal reads occur at the closest preceding synchronization call.

A read observes the value present at that anchor.

3

Signal writes occur at the closest succeeding synchronization call.

That is when the value becomes visible to others.

4

Message passing may shift, share a cycle or overlap…

…within the ordering and boundary constraints on the next slide.

Scope

Sequential processes without loop pipelining. Rolled loops may contain waits. Combinational IO uses settled observations.

Key idea

Everything HLS adds — extra states, pipeline stages, late sampling — must remain explainable by these four rules at the process interfaces.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Here is the basic conceptual model. It covers processes with no loop pipelining; preserved loops are fine, and they may contain wait statements.

Rule 1: synchronization calls within a process always stay in source order.

Rule 2: signal reads occur at the closest preceding synchronization call.

Rule 3: signal writes occur at the closest succeeding synchronization call.

Rule 4: message-passing operations may be shifted in time, issued in the same cycle, or overlapped - subject to a small set of constraints we will look at next.

That is the whole user-level model. Pipelined loops, direct inputs and memories extend it in controlled ways. The thing to remember: whatever HLS adds - extra FSM states, pipeline stages, late physical sampling - must still be explainable by this model at the process interfaces.

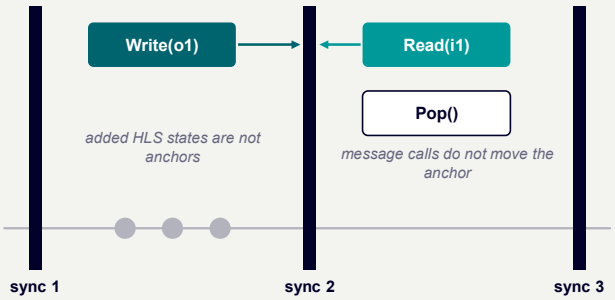
The four-rule summary here is for sequential processes. Combinational processes instead observe stable values after the network settles (R2C).

Signal IO is anchored to synchronization points

```
while (1) {
  o1.write(new_val);       // Write -> next wait
  do {
    wait();
  } while (!go.read());
  new_val = i1.read();     // Read -> prior wait
  new_val = f(new_val);    // no internal IO
}
```

- Read(sig) is anchored to the closest preceding sync call; Write(sig) to the closest succeeding one.
- Several writes before one anchor: the last one supplies the value.
- Same convention as Verilog: inputs sampled at the edge, outputs visible at the next edge.

One process, logical timeline



The physical sample may move; the logical Read or Write does not.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

For cycle-level reasoning, every signal access gets an anchor. A Read is anchored to the closest preceding synchronization call; a Write to the closest succeeding one.

In the code, o1.write is anchored to the wait in the do-while loop that follows it; i1.read is anchored to that same wait, which precedes it. The convention is exactly what Verilog engineers already know: inputs are sampled when the process wakes on the clock edge; outputs become visible at the next edge.

If several writes to the same signal share one succeeding anchor, the last one executed supplies the value; earlier ones are overwritten.

Two consequences matter for HLS. Extra FSM states that HLS inserts are never synchronization calls, so they do not create new anchors. And message operations between a signal access and its anchor do not move the anchor - which, as we will see under RULE 1, is exactly why you should keep signal IO adjacent to its wait.

Message passing: issue order and synchronization boundaries

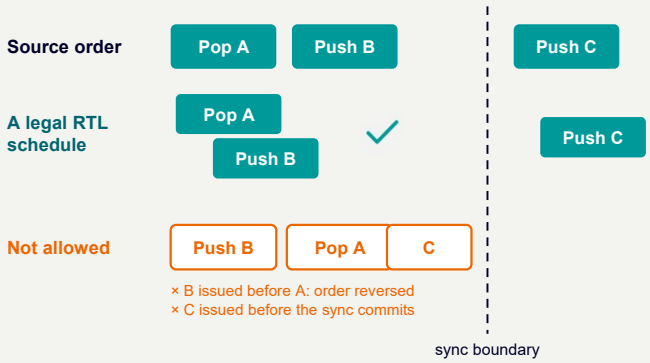
Before a sync
Issued no later than the cycle the sync commits; blocking calls complete by then.

After a sync
Issued only after the cycle in which the sync commits.

Same interface
Issued in source order — also across overlapped pipeline iterations.

Different interfaces
May overlap or share a cycle, but never issued in reverse source order.

Source order versus RTL issue



The sync is a fence: no earlier blocking request survives into the next interval, though a message and its sync may complete on the same edge.

Public | © Siemens 2026 | Siemens Digital Industries Software
SIEMENS

Rule 4 has four constraints.

Before a sync: every message operation in the interval must issue no later than the cycle in which that synchronization call commits - and blocking ones must have committed by then. It is a process-facing completion fence; the message and the sync may complete on the same edge.

After a sync: operations in the following interval issue only after the cycle in which the sync commits.

Same interface: always issued in source order, including across overlapped iterations of a pipelined loop.

Different interfaces: may be issued in the same cycle, and a later call may issue before an earlier one commits, but never in reverse sequence.

In the diagram, overlapping Pop A and Push B is legal. Issuing Push B before Pop A, or letting Push C slip in before the sync commits, is not. One subtlety: a buffered message that already completed at its process-facing boundary may still travel onward externally later.

What the no-reverse rule buys — and what it does not

Guaranteed by the default contract

HLS cannot introduce a new deadlock merely by reversing source-ordered message calls.

Same-interface order and distinct-interface no-reverse order both hold in the RTL, including across overlapped pipeline iterations.

Reversal — the factory deadlock — is ruled out by construction.

Not covered by this rule alone

Loop overlap, eager issue, finite capacities and the environment can still affect deadlock.

The separate completion result (Appendix K) handles these, under its capacity, correspondence and progress premises.

Its conclusion is acceptance of the required workload — not freedom from every internal wait cycle.

`STRICT_IO_SCHEDULING=relaxed`

Lets HLS reorder calls on distinct interfaces (never across sync). May add deadlocks; the formal results assume it is off.

1. **Run relaxed to see the order HLS prefers for QOR**
2. **Rewrite the source in that order**
3. **Return to the default scheduling mode**

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The most important default deadlock-avoidance rule is simple: HLS may overlap or shift message operations in time, but it may not reverse the dynamic source order of message-passing calls. Therefore HLS cannot introduce a new deadlock merely by reversing source calls.

Be precise about the limits. This is not a complete deadlock theorem. Overlapped loop iterations, eager issue, finite channel capacities and environment behavior can still affect deadlock. Those effects are handled by the separate completion result, whose conclusion is that the required workload is accepted - not that every internal wait cycle is gone.

There is a relaxed option, STRICT_IO_SCHEDULING=relaxed, that lets HLS reorder calls freely on distinct interfaces (never across sync calls). It can introduce deadlocks, and the formal results assume it is not used. The recommended loop: run relaxed to see the order HLS prefers for QOR, rewrite the source in that order, and go back to the default.

What HLS may do — and why the interfaces stay compatible

Transformation	What HLS may do	Why interface behavior is preserved
Added states	Insert cycles anywhere in a process, e.g., to share resources	Qualified extra states are silent; values and required observable order are preserved
Loop pipelining	Overlap loop iterations to raise throughput	Stages are internal; channel order, issue order and sync boundaries still hold
Direct inputs	Sample stable signals late, without capture registers	The value is stable, so a late sample equals the value at the anchor
Memory reordering	Move array accesses when proven conflict-free	Logical memory behavior and sync fences are preserved at the declared boundary
Finite buffering	Implement channels with bounded storage	The reference model is configured with the same back-annotated capacities
Cycle-exact modes	`IMPLICIT_FSM=true` or `IO_MODE=FIXED`	Interface cycles identical to pre-HLS; the internal FSM is identical only with IMPLICIT_FSM

Default: HLS may add latency. Cycle-exact modes trade that freedom for identical interface timing.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Each transformation must preserve the qualified interface contract. Added states may be silent steps, but the contract preserves required observable order, not the total listing order of an arbitrary source simulation. Independent events can commute. Timing-sensitive choices retain their own qualification.

Loop overlap preserves source Issue order, channel order and synchronization boundaries under the declared process/boundary evidence. Direct-input sampling preserves the anchored value. Memory transformations stay below a faithful declared boundary. The configured source reference includes actual storage and coupling.

IMPLICIT_FSM=true requires identical interface cycles and internal FSM behavior under its contract. IO_MODE=FIXED requires identical interface cycles while allowing a different internal implementation.

References: rules Additional States, Additional Options, Appendix G; guide §§2–3.

Pipelined loops

Overlapped iterations, initiation interval 1



Orange lines: synchronization calls stay outside the pipelined loop body — before and after it.

No sync inside the loop body

No wait, ac_wait, SyncChannel or ac_sync in a pipelined loop. Put them outside, or don't pipeline that loop.

Message issues stay ordered

Last issue in one iteration ≤ first in the next. In a regular schedule, the message-issue span is at most one II.

Same-interface order stays intact

Across iterations, preserve access order on each message, signal or sync interface. Signal IO keeps its anchors.

Changes are flagged

If pipelining would reorder signal IO or split same-edge IO, the tool requires an explicit pragma (Cat7).

Pipelining: preserve the qualified interface contract. Finite storage and controller progress still need their own evidence.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The timeline shows iterations overlapping in physical pipeline stages. It does not assign source-level message calls to those stages, and it does not by itself establish a sustained initiation interval of one cycle.

An ordinary pipelined loop body contains no explicit wait, ac_wait, SyncChannel or ac_sync. Place synchronization before and after the loop. Signal accesses remain tied to those real synchronization points, even when the physical accesses occur in different pipeline stages.

Source message requests remain ordered across iterations. The last message issue of iteration j cannot follow the first message issue of iteration j+1. In a regular repeating issue schedule, the span from an iteration's first to last message issue is therefore at most one initiation interval. The boundary may share a cycle. This limits message issues, not the entire computation or all completions (E3).

The same-interface rule is broader: accesses to each message, signal or synchronization interface keep their order across iterations. This does not authorize synchronization inside the pipelined body.

A physical FIFO read or forwarding step is not automatically the source Pop's issue or completion. The interface mapping must identify those events, and storage holding accepted messages still counts as communication storage (B.BoundaryBinding).

If pipelining would reorder signal IO or split IO that shared a source edge, the tool requires explicit authorization (Cat7). That authorization does not waive the observation rules or the separate controller, capacity and progress evidence.

(References: Pipelined Loops, Appendix B, E3, Core.RegSeq and guide §9.)

Pipelines must flush, not freeze

Global stall: fails the B.OF profile



Everything stops: completed downstream work cannot leave while upstream is blocked.

Ordered automatic flush (B.OF)



Once blocking restrictions begin, no new later work launches. Completed downstream work can still drain.

When blocking begins

After signals settle, none of the earliest pending blocking operations can proceed.

No new later work

No younger iteration launches or issues beyond the blocked operations.

No withdrawal

Issued requests persist until commit or reset. Pre-issued work may still complete.

Ordered drain

With downstream support and service, completed work drains despite upstream blocking.

`hls_stall_mode flush` is a request, not evidence: the generated controller must actually meet B.OF / B.CT.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The blocking restrictions begin at the first settled snapshot where there are pending blocking operations at the earliest unfinished source positions and none can proceed. These operations may form a group rather than a single call (the pending blocking frontier, Core.16).

While that condition persists, no new source-later evaluation, preparation or request may start beyond those operations. Requests legally started during the earlier preparation window are not retroactively forbidden.

Two kinds of activity may continue. Work already admitted downstream may drain. A later request issued legally before blocking began may also complete, provided it preserves the operands, descriptors, storage and effects still needed by the stalled older work. An affecting reset follows its declared treatment of outstanding work.

For completion, the controller must do more than obey the restrictions. Completed downstream work that is able to advance must receive ordered opportunities to leave, independently of upstream blocking or new input, under the declared downstream and service conditions (B.OF). This does not eliminate downstream backpressure or require bubble compression: completed work behind incomplete downstream work may still be held (B.OF.Drain).

A global stall that indefinitely freezes required completed downstream work fails this completion profile. A flush pragma requests the behavior; evidence about the generated controller must establish it. An implementation outside the standard profile needs a separate proof that required requests become possible and progress.

(References: Core.16, Appendix B B.OF/B.CT and guide §§3 and 9.)

Direct inputs: stable signals, sampled late

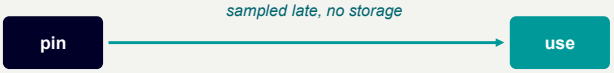
Ordinary signal read



Preserve the anchored value through added states or stages. This can require capture and pipeline storage.

```
#pragma hls_direct_input
```

sampled late, no storage



Logically, the Read still happens at its anchor. Physically, HLS may sample the stable signal whenever it needs the value.

The contract

The environment holds the signal stable once the receiving processes leave reset, until the next reset affecting them.

The freedom

HLS samples physically as late as it likes, with no capture registers — saving area.

The observation

Late samples are internal: no extra Read events, and the attributed value cannot change.

Dynamic resets

Allowed. Each reset starts a new stability epoch for the affected logic and its direct inputs.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Ordinary signal reads happen at the closest preceding synchronization call, in both models. So if HLS adds states or pipelines the process, it must preserve the anchored value until it is consumed, which can require capture and pipeline storage. With many signals that is expensive, and often unnecessary.

The simplest fix is for signals that are held stable after the process comes out of reset - configuration registers, for example. Mark them with pragma hls_direct_input. HLS may then sample the physical signal as late as it likes with no register storage, while for scheduling and source/RTL comparison the Read is still treated as happening at its anchor.

The obligation moves to the environment: it must hold the signal stable within each reset epoch. Dynamic resets are fine; each reset starts a new stability epoch.

Later physical samples are internal implementation activity: they create no extra Read events and cannot change the value attributed to the logical Read.

Changing configuration safely: hls_direct_input_sync

```
#pragma hls_direct_input
sc_in<uint32_t> direct_inputs[N];
...
while (1) {
  #pragma hls_direct_input_sync all
  sync_in.sync_in();     // update point
  #pragma hls_pipeline_init_interval 1
  #pragma hls_stall_mode flush
  for (x = 0; x < direct_inputs[0]; x++)
    for (y = 0; y < direct_inputs[1]; y++) {
      sum = 0;
      for (s = 0; s < num_samples; s++)
        sum += sample_in[s].Pop()
               * direct_inputs[2 + s];
      ...  out1.Push(result);
    }
}
```

Abridged from Catapult Matchlib example 61

Only sync may gate these updates — never Push/Pop or ac_channel.

1. **DUT asserts and holds sync rdy**

   Only once old values are no longer needed — pipeline ramped down, or remaining uses protected.

2. **TB changes inputs, asserts vld**

   After seeing rdy; the new values settle before the sampling edge.

3. **Sync completes at the edge**

   With rdy held high, one cycle of vld is enough if that edge completes the handshake. Hold offers until commit or reset.

4. **Inputs stay stable**

   Until the next permitted update or an affecting reset.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Sometimes control inputs must change while the design runs - for example, an image pipeline whose processing parameters change at the start of each frame. Before the inputs change, every remaining use of the old values must be finished or protected. Ramping down the pipeline is one way to do that; it is not required when the remaining old operands are retained and isolated from the pin change. Separately, capacity for next-frame work is withheld while the DUT is pending at the sync (Core.16(7)), but that admission control does not itself protect old-value users.

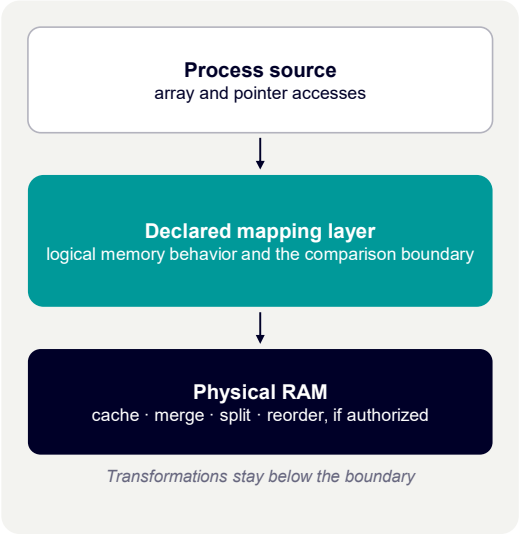
The mechanism is a SyncChannel or ac_sync call, tied to the direct inputs it controls by pragma hls_direct_input_sync. This is Catapult example 61.

The handshake: the testbench waits for the DUT's sync ready. By asserting ready, the DUT has finished with the old values or protected any remaining uses. The testbench changes the inputs and asserts sync valid, letting them settle before the sampling edge. With ready still high, the sync completes on that edge, so one cycle of valid is enough - but do not drop valid just because a cycle passed without a handshake. Then keep the new values stable until the next permitted update or reset.

Only the sync operation may gate these updates - not Push, Pop or ac_channel. If the environment needs to decide when to update, send a separate request asking the DUT to call sync; that request is not itself permission to change the inputs.

Both sync endpoints retain their offered signal and occurrence identity through commit or an affecting reset. Old-value protection begins before ready is asserted and remains valid while the offer is pending and for any later protected use. All eligibility and fence conditions act before the sampled overlap.

Arrays and memories



Declare the mapping

Name the logical memory behavior and the boundary at which source and RTL are compared.

Transform only below it

No visible interaction created, dropped, duplicated, relabeled or illegally reordered.

Respect sync fences

Accesses before a sync commit no later than the sync; never merge accesses across one.

Expose cross-process values

Use covered messages or anchored signals for shared values (J.Mem). Sync orders accesses but does not carry values.

Count backpressure

A memory that can stall a process belongs in the completion qualification.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Arrays may remain arrays through synthesis and map to RAMs. Pointer accesses resolve to array accesses. Each mapped memory needs a declared description of its logical behavior and the boundary at which source and RTL are compared.

Below that boundary, authorized caching, merging, splitting and reordering may change physical RAM traffic. They must not create, drop, duplicate, relabel or illegally reorder interactions visible at the comparison boundary.

Synchronization still fences accesses. Earlier accesses commit no later than the sync, later accesses commit after it, and one merged commit cannot represent accesses on both sides.

When one process's write affects another process's values, decisions or operation order, the model must expose that dependency. The supplied proof route represents these values through covered messages and/or signals tied to synchronization points (J.Mem). Synchronization orders accesses but does not itself carry the written value or prove equal read values.

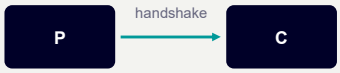
Reusable shared-memory wrappers, such as qualified ping-pong memories or native RAM FIFOs, can package this evidence. A more general proof based directly on shared memory state needs a separately supplied extension. A memory that stalls a process also needs to be included in the completion argument.

(References: Scheduling of Array Accesses, J.Mem and Core.MemFaithful.)

Channel capacity belongs in the reference model

Rendezvous

`B(c) = 0`



Push and Pop commit together: same cycle, same value.

Bounded FIFO

`B(c) > 0`



Pop needs an item at the start of the cycle; Push needs space then. No fall-through within a cycle.

Elaborated structure

joins, staging, gates



Explicit finite storage plus stateless ready/valid couplers, when one depth is not faithful.

The reference model is not the raw simulation

Raw SystemC
unadjusted channel depths

≠

Reference model Sys_ref
source + testbench with back-annotated capacities, arbitration and coupling

Matchlib note

In<> ports default to a one-place skid buffer for throughput modeling; check your version.

For final verification, match ports and capacities to the selected RTL and re-establish faithful capacities, e.g. with

`-DFORCE_AUTO_PORT=Connections::SYN_PORT`

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Channels come in two basic flavors.

Capacity zero is a rendezvous: the matching Push and Pop commit together in the same cycle with the same value.

Positive capacity is a bounded FIFO with cycle-boundary, non-fall-through semantics. A Pop needs an item present at the start of the cycle; a Push needs free space at the start of the cycle. A full FIFO cannot accept a Push just because a Pop happens on the same edge, and an empty FIFO cannot satisfy a Pop with that edge's new Push.

When a single depth does not describe the structure - joins, staging, coupled multi-input pipelines, sync gates - the reference model is elaborated with explicit storage plus stateless ready/valid couplers.

The important consequence: the reference model is not the raw SystemC simulation. It is the source and testbench configured with the implementation's back-annotated capacities and communication structure. Matchlib In ports model a one-place skid buffer by default for throughput modeling. Check that description against the library version and configuration in use; it does not by itself establish throughput accuracy. For final verification, configure ports and capacities to match the selected RTL.

Matching capacity alone does not establish completion.

RULE 1: every signal access needs one unambiguous anchor

Risky **A blocking Pop splits read from wait**

```
do { wait(); } while (!go.read());
int pop_val = pop1.Pop(); // may stall
new_val = i1.read();  // i1 may have
                      // changed by now
```

Fix: read i1 immediately after its wait. No cost in QOR or simulation speed.

Ill-formed **One write, two possible anchors**

```
wait();                  // WAIT 1
o1.write(some_value);
if (go.read()) { ... }
else { wait(); }         // WAIT 2
some_other_function();
wait();                  // WAIT 3
```

o1 binds to WAIT 2 or WAIT 3 depending on go. Flag as an error; move the write next to its intended wait.

RULE 1 Keep signal IO next to its wait. Whether an observable read occurs, and its value, must be determined at its anchor. Resolve or explicitly justify violations.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

For sequential processes, keep signal IO next to its synchronization call. In the left example, raw simulation may sample i1 after a stalled Pop even though the logical read belongs to the preceding wait. Moving the read next to that wait removes this mismatch risk. The right example has two possible following anchors for its write and needs a rewrite or an explicit justification allowed by the rules.

For a read included in the observable interface history, both whether it occurs and its value must be determined at its anchor. Consider wait(); x = in.Pop(); if (x) y = cfg.read();. A stable cfg value is not enough: the later Pop result decides whether the read exists. It is too late to add that read to a synchronization group that has already completed. Conditional reads are allowed when the decision is already determined at the anchor (retained-Read qualification).

There is a separately qualified way to observe configuration updates for designated direct inputs instead of observing ordinary reads (configuration-observation route). It has its own conditions and does not waive the ordinary read rules.

Resolve the diagnostic, supply the permitted explicit justification, or exclude the unsupported case from the guarantee.

(References: RULE 1, Core.17 and guide §5.)

Rules: RULE 2 (Cat4)

RULE 2: signal IO inside rolled loops

```
wait();             // start of while loop
#pragma hls_unroll no
for (int i = 0; i < 10; i++) {
  o1.write(i1.read() * i); // no anchor
}                          // in the loop
```

Why it is a problem

- With no sync inside the loop, every iteration shares one pair of anchors.
- Logically there is one Read; only the final Write supplies the value at the next anchor.
- An RTL that samples a fresh i1 per iteration differs from the anchored meaning. Added states never create anchors.

Per-iteration IO, loop not pipelined

Place a wait() in the loop body — and follow RULE 1.

One value intended

Move the read before the loop to make the intent explicit.

Loop will be pipelined

No in-body wait allowed: move IO outside, use message passing, use hls_direct_input(_sync), or don't pipeline.

RULE 2 Signal IO in a rolled loop needs a wait in the loop body.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

RULE 2 covers signal IO inside rolled, or preserved, loops.

In this example the loop body reads i1 and writes o1 ten times, but there is no synchronization call inside the loop. So every iteration shares the same anchors: logically, one read at the preceding wait, and only the final write supplies the value at the succeeding anchor. An RTL that samples a fresh i1 on each iteration across added clock cycles - or exposes each o1 value - differs from both the raw zero-time simulation and the anchored meaning. Added implementation states never create new anchors.

So decide what you meant. If you want per-iteration interaction and the loop is not pipelined, put a wait in the loop body and follow RULE 1. If you meant one value, read it before the loop. If the loop is to be pipelined, an in-body wait is not allowed: move the IO out, use message passing, use direct inputs, or do not pipeline that loop.

As with RULE 1, a warning must be resolved, explicitly justified, or treated as excluding the model from the formal guarantees.

Three common sources of pre-HLS / post-HLS mismatch

Non-blocking calls

PushNB / PopNB / nb_read / nb_write outcomes depend on latency — which HLS changes.

Instead

Use only when needed, e.g., time-based arbitration at a shared resource.

One-way messages as sync

Latency and storage between processes vary, so one channel does not synchronize two processes.

Instead

Use SyncChannel, or two opposite channels (AXI ar/r, aw/b) — weaker, but valid.

Signals without a handshake

wait()-synchronized signal IO without a two-way handshake is likely latency-sensitive.

Instead: a two-way handshake

```
out_dat = value;
out_vld = 1;
do {
  wait();
} while (out_rdy != 1);
out_vld = 0;
```

Rule of thumb: if behavior depends on how long something took, it is latency-sensitive — synchronize explicitly or isolate it.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

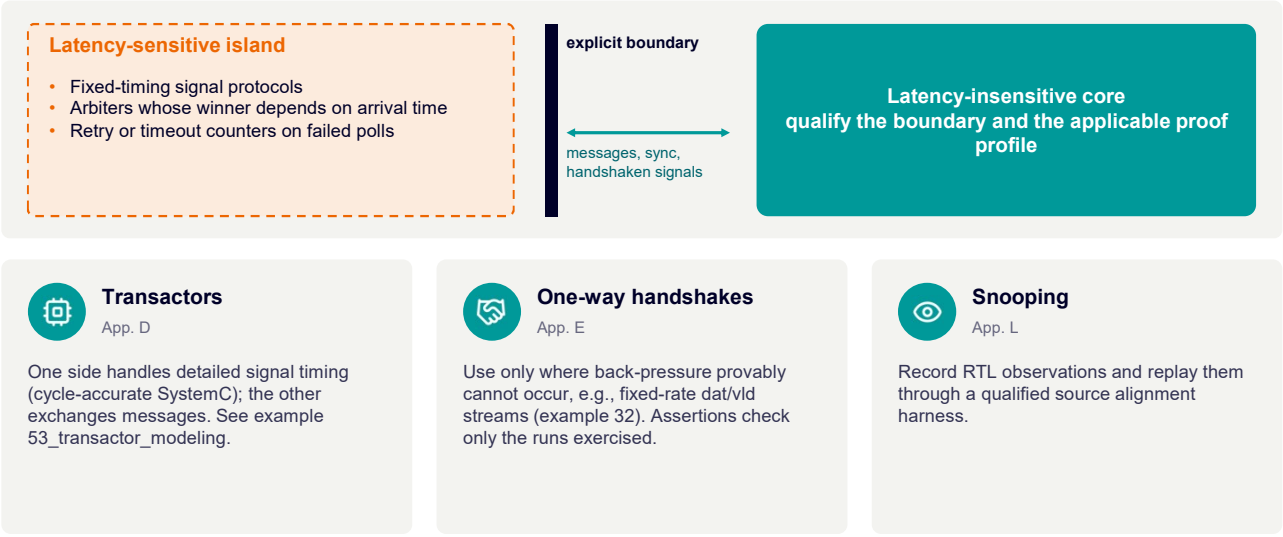
Three patterns cause most mismatches between pre-HLS and post-HLS simulation.

First, non-blocking message calls - PushNB, PopNB, nb_read, nb_write. Their outcome depends on latency, and latency is exactly what HLS changes. Use them only when nothing else works - time-based arbitration of several streams at a shared resource is the classic legitimate case.

Second, using a one-way message channel as if it synchronized two processes. Message latency and storage between processes vary, so it does not. Two channels in opposite directions can synchronize - think AXI ar and r, or aw and b - though that is weaker than an explicit SyncChannel or a two-way signal handshake.

Third, signal IO synchronized with a wait but without a proper two-way handshake. That is likely latency-sensitive. Use a handshake like the one shown; protocols such as APB use different but still latency-insensitive handshakes.

Isolating latency-sensitive behavior



 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Timing-sensitive protocols, arrival-dependent arbitration and retry or timeout logic need explicit interface boundaries and supporting evidence. Isolating them is useful architecture, but does not automatically prove safety or completion.

Transactors use cycle-accurate signal behavior on one side and messages on the other. Example 53 illustrates this style. A one-way handshake needs an established no-backpressure property over the qualified operating, stall and reset conditions. Runtime assertions monitor only the runs actually exercised; a clean finite run does not prove that property.

Snooping can make a source run follow choices observed in a particular RTL run. The checking harness records each relevant RTL occurrence and supplies those observations to the source in order. It checks occurrence identity, operation kind, payload and any outstanding events. Simply AND-gating live signals can lose or confuse events when the source lags.

Keep the RTL free-running. Any throttling of the source must remain legal under its original protocol, and an issued source request must persist until commit or reset. The harness cannot suppress a mandatory handshake, invent a non-blocking failure or substitute a different payload (Appendix L harness requirements, L.Dir and W1–W5).

Snooping is one available technique, not a universal requirement. Matching one observed RTL run does not prove that all executions needed by the workload-completion claim make the same relevant choices and consume the same values. Nor does it establish progress.

(References: Appendix L, L.WC and guide §10.)

Rules: Appendix F

Modeling guidelines at a glance

In general

- Prefer `Connections::In/Out` + `SyncChannel` over signal IO and wait()
- Prefer `Pop()/Push()` over `PopNB()/PushNB()`
- For pipelined loops in completion claims, check that the generated controller meets the flush laws
- Unless modeling cycle-accurately, expect latency to differ between pre- and post-HLS models

When using signal IO

- Cycle-accurate process: use `disable_spawn` and the example-53 transactor style, with no Push/Pop
- Direct inputs: `hls_direct_input` (+ `_sync`); change only at allowed times
- Mixing signals with In/Out: use a real signal handshake, not the ports, for synchronization
- Keep signal IO right next to its wait()

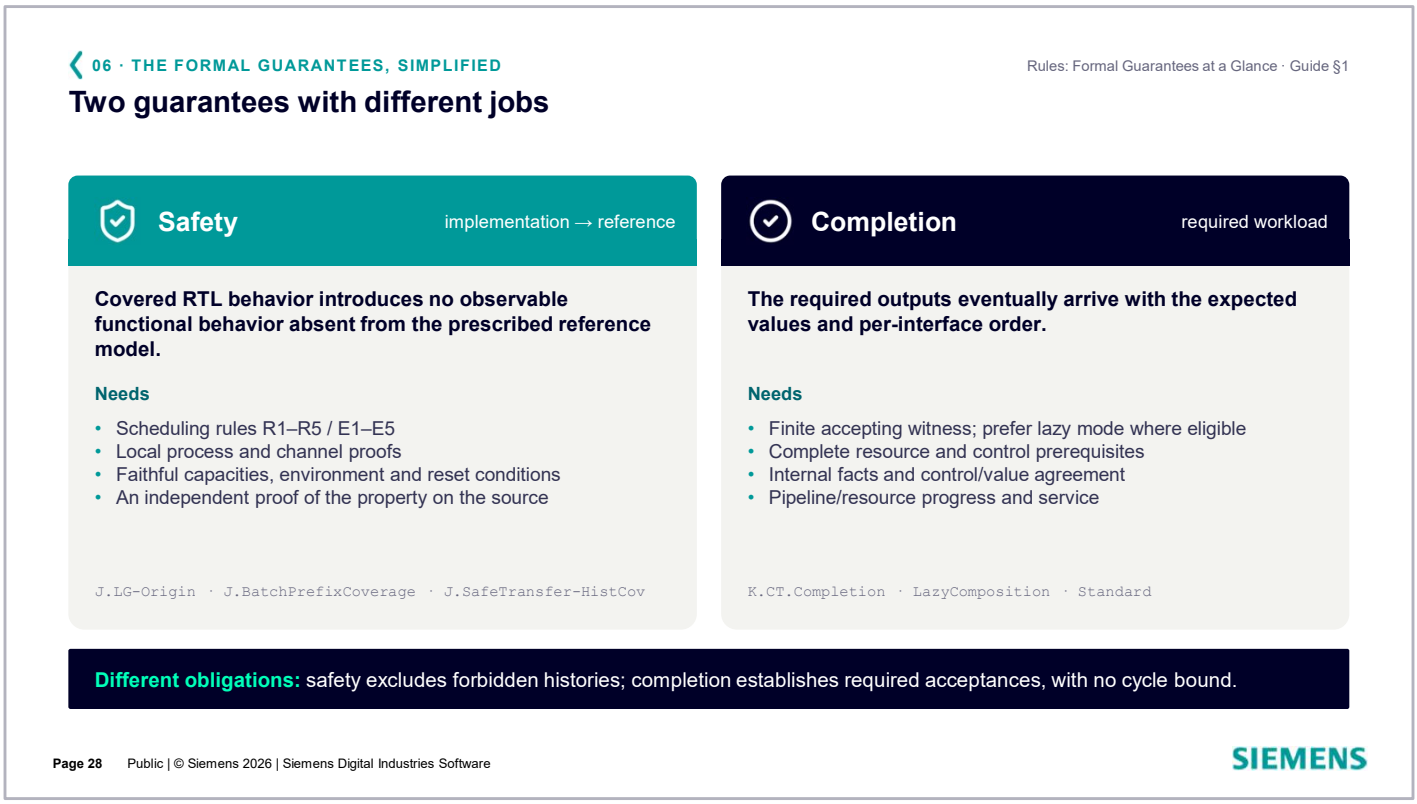
 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Appendix F condenses the modeling practice into a short list.

In general: prefer Connections In and Out ports plus SyncChannel over raw signal IO and wait. Prefer blocking Pop and Push over the non-blocking variants. For pipelined loops used in completion claims, check that the generated controller really meets the flush laws - the directive alone is not evidence. And unless you are modeling a cycle-accurate process, expect latency to differ between the pre-HLS and post-HLS models; write your testbench accordingly.

When you do use signal IO: for a cycle-accurate process, use disable_spawn and the transactor style of example 53, with no Push or Pop in that process. For direct inputs, use hls_direct_input and possibly hls_direct_input_sync, and change the signal only at allowed times. When mixing signal IO with Connections ports in one process, use a proper signal handshake that does not rely on the ports for synchronization, and keep the signal IO right next to its wait.



The formal architecture separates safety from liveness, and it is worth keeping them apart in your head.

Safety runs from the implementation to the reference: covered RTL behavior introduces no observable functional behavior that is absent from the prescribed reference model. It is largely compositional - local evidence for processes, channels, atomic interactions, environment and reset is combined by a generic theorem. Its payoff is that properties established at the source hold on the RTL.

Completion is about getting work done: the required outputs eventually arrive with the expected values and per-interface order. It needs more: a finite accepting source run checked against the proof's resource and control prerequisites, correspondence of internal facts, pipeline and resource progress, and service or fairness assumptions.

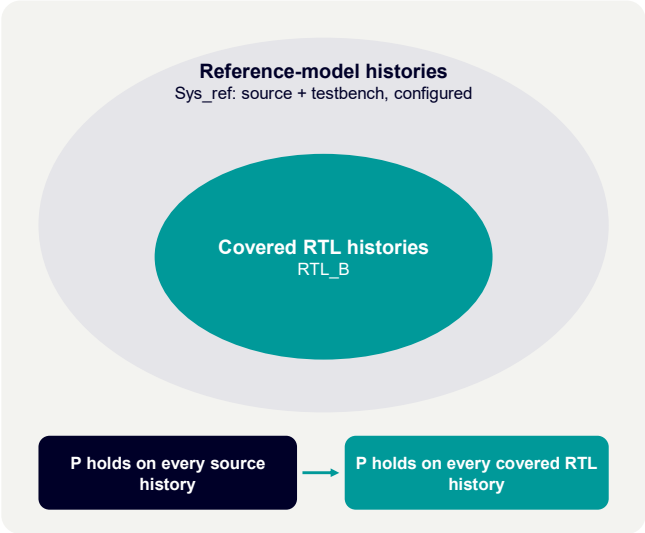
Neither implies the other. Safety does not promise any output at all; completion does not promise a cycle bound. Extra outputs are excluded by safety, not by completion.

The witness may be ordinary serial or a qualified group-serial witness for indivisible atomic groups. Completion agreement includes operation identities and consumed control decisions, not payloads alone.

A passing serial (Policy S) run is the starting point, not the whole check. The standard completion proof (Appendix K) also requires its complete prerequisite graph, including guaranteed-space dependencies, to be cycle-free; the completion workflow later in the talk explains this check (K.CT.SerialWitness, K.CT.Closure).

Rules: App. G · J.ReachableHistCov · J.SafeTransfer-HistCov

Safety: every covered RTL behavior has a source explanation



Direction matters

RTL → source. A particular pre-HLS run is not guaranteed to be reproduced by the RTL.

"Equal" means equal history

Same events, values, required order and indivisible groups. Independent events may occur in different cycles.

Write properties on that history

Check accepted values and required order, observing whole atomic groups. Exact cycle-count checks do not transfer.

The source property is separate

It must hold for all admitted source behaviors. One passing simulation is not that proof.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Think of the interface histories that each model can produce. For every finite covered RTL execution, the safety proof constructs a legal reachable source execution with the same selected observable history. The direction is RTL to source. It does not promise that the RTL reproduces an arbitrarily chosen source simulation.

Equal history means equal events, values and required order at the selected interfaces. Independent events may occur in different cycles or be logged in a different order. Complete atomic groups remain together.

A scoreboard that checks accepted values and required request/response order can carry over if it uses only this history. A check requiring a response on an exact cycle generally cannot. Nor can a checker depend on which callback happens to run first for independent events, or inspect an intermediate logger state partway through an indivisible group (Batch observation profile).

The source property must separately hold for every admitted source behavior. One passing simulation does not establish that universal property. Once the source proof and the correspondence evidence are supplied, the property also holds on covered RTL executions.

Source issue order on different interfaces does not automatically create a strict order between their completions.

(References: Appendix G, J.ReachableHistCov and J.SafeTransfer-HistCov.)

Rules: Appendix G

The trace rules: R1–R5 for the source, E1–E5 for the RTL

Topic	Source rule (pre-HLS reference)	Implementation rule (post-HLS RTL)
Sync order	R1 · Sync calls occur in program order	E1 · Per-process sync order preserved
Signal anchoring	R2 · Reads at the preceding sync, writes at the succeeding sync. R2C: combinational logic observed at its settled fixed point	E2 · Signal visibility preserved at the same anchors
Isolation around sync	R3 · Messages stay in their interval; blocking ones complete by the sync	E5 · Messages cannot cross their immediate sync boundary
Message issue order	R4 · Never issued in reverse source order	E3 · Same ordering, prefix-closed, also across overlapped iterations
Channel semantics	R5 · Rendezvous (B = 0) or bounded FIFO (B > 0)	E4 · No drops, duplicates or reordering; occupancy stays within 0…B

τ_ref ≈ τ_post is an ordered compatibility relation — source witness on the left, RTL on the right — not a symmetric equivalence.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Appendix G restates the contract as trace rules. The R rules describe the source reference; the E rules are the matching obligations the implementation must preserve.

Synchronization order: R1 and E1 - sync calls stay in program order within each process.

Signal anchoring: R2 and E2 - reads at the preceding sync, writes at the succeeding sync. Combinational processes are instead observed after their signals settle each cycle (R2C).

Isolation around sync: R3 and E5 - messages cannot cross their immediate synchronization boundary, and blocking ones complete by the sync.

Message issue order: R4 and E3 - never reverse source order. The rule applies throughout execution: if a later call has issued, every earlier one has too, including across overlapped iterations.

Channel semantics: R5 and E4 - rendezvous or bounded FIFO, no drops, duplicates or reordering, occupancy always between zero and the capacity.

Note the compatibility relation is ordered - source witness on the left, RTL on the right. It is not claimed to be a symmetric equivalence.

Rules: App. J · Guide §4

How local proofs compose into a system result

1 Local evidence

Per process, channel and atomic interaction: source and RTL steps correspond.

`contracts C0–C5`

2 Integration checks

The ready/valid network settles; environment, reset and capacities are faithful.

`settling · environment`

3 Generic composition

Induction over actual RTL steps builds a legal source prefix for every RTL prefix.

`J.LG-Origin · J.BatchPrefixCoverage`

4 Safety transfer

A property proved separately on the source carries over to the RTL.

`J.SafeTransfer-HistCov`

Why it matters

Local evidence replaces an intractable whole-system proof. For ordinary profiles, no design-specific global invariant is needed (J.OrdinaryConstructiveComposition).

What it does not assume

No fairness and no future cycles: deadlocked and terminated prefixes are covered too. It gives safety only — no progress conclusion.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The safety proof builds a system argument from local evidence.

First, establish how each process, channel and atomic interaction implements its source behavior. The registered message profile states these local requirements explicitly (contracts C0–C5).

Next, check the integration: the ready/valid network settles, and the environment, reset behavior and capacities match the model.

The generic theorem then follows the actual RTL steps and constructs matching legal source steps. It covers every finite prefix and preserves complete observable groups (J.ConstructiveEdgeInduction, J.LG-Origin and J.BatchPrefixCoverage).

Finally, a property proved independently for all admitted source histories also holds for covered RTL histories (J.SafeTransfer-HistCov).

The benefit is reusable process-by-process and channel-by-channel evidence. For the ordinary profiles, the composition does not require an additional design-specific global invariant beyond its stated premises. It also does not assume future progress or fairness, so its safety conclusion includes executions that stop or deadlock.

(References: Appendix J and guide safety construction.)

Testbenches, properties and stimulus classes

The testbench is in the system

Reactive responses that depend on earlier DUT results are covered by the same joint argument.

One proof for a class of stimuli

A reusable correspondence proof can cover a declared class of inputs and choices; each new stimulus needs only binding and membership checks.

The source property is your job

It must hold for all admitted source behaviors. A passing sample campaign does not establish that.

Latency-sensitive testbenches

Branches, watchdogs or pass/fail decisions on raw cycle counts are not covered automatically: isolate, make explicit, or check at RTL.

Theorems: J.UniformConstructiveCoverage, J.ConstructiveClassSafe. Reuse can cut effort across a large, stable campaign — with no unconditional cost advantage.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

This slide is for the DV engineers in the room.

The testbench is normally included with the design in the compared system. That means reactive responses - stimulus that depends on earlier DUT results - are covered by the same joint argument. The implementation-side testbench need not be synthesized, but its runtime and interface semantics must be qualified.

One reusable correspondence proof can cover a declared class of stimuli and choices, not just a single test. Once it is established uniformly, each admitted stimulus needs only exact instance binding and membership checks.

But the source property is still your job. It must hold for all admitted source behaviors. A passing sample campaign or a shared testbench program does not establish that.

And latency-sensitive testbenches are a special case: branches, watchdogs or pass/fail decisions that depend on raw cycle counts are not automatically covered. Isolate them, make them explicit, or check them at RTL.

Reuse can reduce total effort across a large, stable campaign, but the rules are careful to claim no unconditional cost advantage.

The reusable stimulus-class correspondence discussed here is safety evidence. Required-workload completion still needs an accepting source witness and its certificate for the workload being claimed (K.CT.Completion).

Rules: App. K (K.CT) · App. N · Guide §8

Completion: proving the required outputs arrive

1 Declare the workload
The testbench acceptances you require, with values and per-endpoint order.

2 Run a qualified witness
Real capacities; prefer lazy Policy S where eligible. Qualified atomic groups use group-serial issue.

3 Establish the certificate
Include all prerequisites. Use restricted rank evidence or check the assembled graph's acyclicity.

4 Apply correspondence
Correspondence and agreement cover consumed facts, decisions and values; not an RTL trace match.

5 Show progress
Qualify controller and resource exposure. Keep clocks and required environment service running.

`K.CT.Completion` Every qualified RTL execution that satisfies the service assumptions eventually accepts the workload with the same values and per-endpoint order.

Serial = conservative
Within a sync interval, each blocking call issues in a later cycle than all earlier blocking commits.

Failure is inconclusive
A failed witness or certificate is not a deadlock verdict.

Not claimed
No cycle bound; unrelated or post-completion deadlocks are outside the result.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Declare required testbench acceptances, their values and per-endpoint order. A DUT offer alone is not acceptance. The witness is a finite legal run of source plus testbench using the implementation-bound reference. It need not drain unrelated work.

The default workflow is Policy S: each owner's later blocking operation issues in a later cycle than earlier blocking commits. Independent owners may run concurrently. Qualified atomic inputs use complete group-serial batches, not individually serialized members.

The complete prerequisite inventory includes owner/endpoint order, older-stage and older-iteration work, preparation and CtrlSupport, including entry, branch and negative required-work decisions. The resulting dependency graph must be acyclic: the lazy route derives this from its rank lemmas, and the standard route checks the graph including its floor edges.

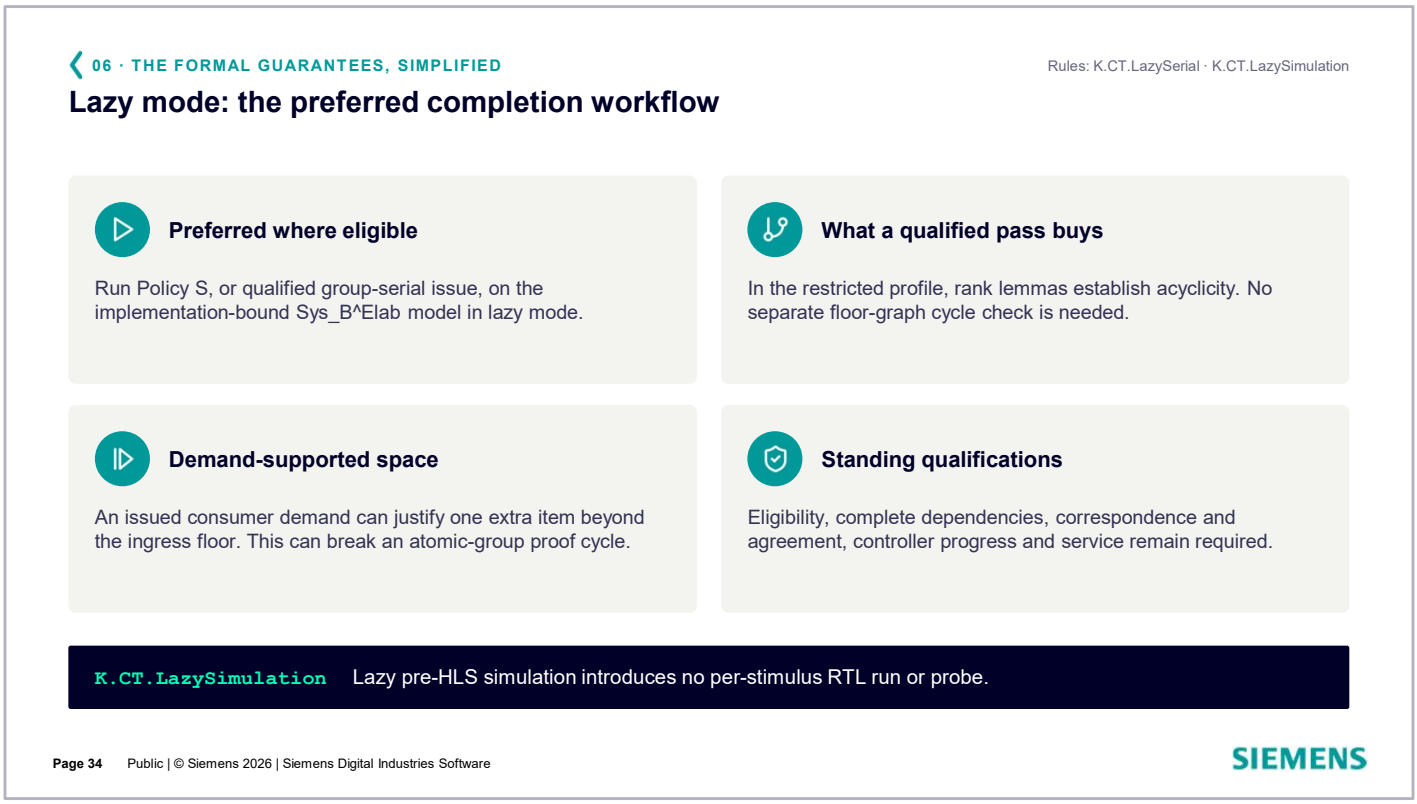
Apply correspondence and agreement evidence to all consumed operation identities, facts, decisions and values. This does not mean matching one source run to an observed RTL simulation. Timing-dependent choices retain their independent agreement obligations.

The following four slides explain lazy mode, the preferred way to run this workflow where eligible. The standard floor route and its capacity example follow them. Ordinary selected-channel Push cases retain standard floor resources; only qualified extra-slack cases use demand support.

Controller and resource proofs must establish exposure under all admitted surrounding activity. Physical settled ready and valid force transfer at the applicable edge. Required clocks and environment service continue until the workload is accepted. Reset, cancellation or premature test termination cannot erase required work without preservation evidence.

The conclusion is eventual acceptance with the witness values and per-endpoint order, not a cycle bound or total deadlock freedom. A failed witness or sufficient certificate is inconclusive.

References: K.CT.Instance, Witness, CtrlSupport, Facts, Agreement, Completion, LazySerial, LazyComposition, LazySerialRestricted; Appendix N; guide §8.



Prefer the default serial workflow in lazy mode on eligible channels. This is the same implementation-bound permissive reference used for safety, with a restriction on which optional abstract moves the witness selects. Qualified atomic batches retain group-serial issue.

In the restricted profile, a qualified lazy pass supplies the acyclicity argument: LazyRank/LazyMixedRank order all edges in the prescribed complete certificate. There is no separate floor-cycle test that can reject such a qualified witness. This is not the slogan "a pass is the certificate": eligibility, complete support, fact coverage, agreement, actual witness membership and exposure still need evidence.

K.CT.LazyComposition is the general mixed-component theorem. It keeps unselected units' qualified proofs and every cross-component dependency, and requires acyclicity proved or soundly checked on the complete combined graph. K.CT.LazyExtendedStandard separately returns to the standard route after a protected singleton extension. A failed lazy run is inconclusive and may be conservative. It exposes a source-execution obstruction, not necessarily RTL deadlock.

An issued demand may justify one staged outstanding item beyond the ingress floor when credit allows it. The benefit is particularly useful for atomic input groups, explained in the proof slide's notes. It does not justify full use of F+m storage.

No per-stimulus RTL simulation or probe is introduced. Capacities, topology and boundary mapping come from structural binding. Reusable implementation conformance and instance checks remain. Any independently selected snooping dependency remains separate.

References: K.CT.LazySerialScope, LazySimulation, LazySerialRestricted, LazyComposition, LazyExtendedStandard; Formal Guarantees at a Glance.

Rules: K.CT.LazySerial · K.CT.LazySimulation

What lazy mode changes

One input channel in the elaborated model



Outer boundary
Push valid/ready unchanged.

Optional moves
Only while Pop(j) is issued and unretired.

Process-facing Pop
Demand exposed independently of arrival.

Teal: occupied Dashed: empty **Orange: move gated by demand**

The source demand
Expose Pop(j) once legally reached and prepared, independently of whether its item has arrived.

The internal movement
Move item j through optional abstract staging only while its process-facing source Pop(j) is issued and unretired.

The outer boundary
The producer holds valid until ready. It never waits for ready before asserting valid.

What stays the same
Source Push/Pop calls, capacities and mandatory physical handshakes retain their declared semantics.

Optional abstract moves only. A physical staging handshake cannot be suppressed.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The demand belongs to the consumer process in the pre-HLS simulation. It is not an RTL request and is not the early movement at the outer ingress boundary. Once the Pop is legally reached and prepared and its issue is allowed by the issue/phase discipline, expose and record the request independently of matching-item availability. For a group, expose the complete legally prepared batch. Core.RegSeq and Core.16 still apply. Demand need not precede arrival if data is already present.

The lazy selector permits the ingress-to-staging move, and further optional moves of item j, only while source Pop(j) is issued and unretired. The producer still asserts and holds valid and payload normally; there is no ready-before-valid addition.

This applies only to optional abstract moves. An internal boundary declared as a physical ready/valid interface obeys its mandatory handshake law and cannot be suppressed. A skid buffer or register slice must be classified by its bound semantics, including any Matchlib In<> realization. At-site split capture into attributed slots remains outside this profile.

Select jointly legal lazy moves promptly with a deterministic rule, while preserving old-state, simultaneous-update and conflict constraints. One item cannot cross multiple locations on a single edge contrary to the component law. Prompt selection is practical guidance, not a general schedule-dominance theorem or new hardware progress premise.

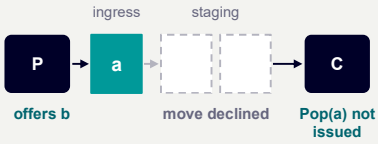
References: K.CT.LazySerialWitness, LazySimulation.DemandVisibility/Selection; B.BoundaryBinding.

Rules: K.CT.LazySerial · K.CT.LazySimulation

Example: one ingress entry, two staging locations

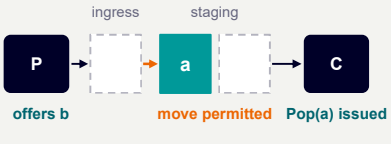
Independent staging: F = 1 ingress entry, m = 2 staging locations, no tighter credit cap

1 Before Pop(a) issues



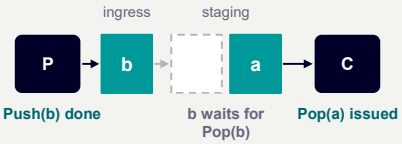
Item a stays in ingress, even though staging is empty. The producer may offer b and remain pending normally.

2 Pop(a) is issued



The pre-HLS consumer exposes its legal demand. Permitted moves of a can now release ingress space.

3 Item b is admitted



The producer can complete Push(b). Item b stays out of staging until its own process-facing Pop(b) issues.

`LazyInventory` At most F + 1 = 2 unmatched items here. Physical capacity remains F + m = 3.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Initially a occupies the single ingress entry and both staging locations are empty. In permissive normal mode, the reference may move a into staging before its consumer Pop is issued. In lazy mode it declines that optional move until the source process-facing Pop(a) has legally issued.

The producer may already be offering b. It does not withhold valid while awaiting ready. After Pop(a) issues, permitted movement of a releases ingress space. With old-state admission, b can enter only after that release has taken effect. Moves continue according to the declared per-edge component laws; no instantaneous two-stage traversal is assumed.

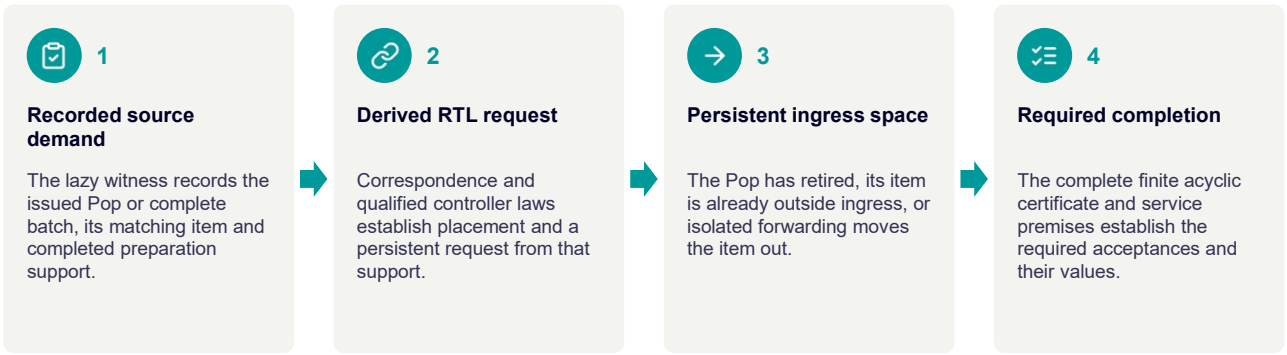
Item b cannot enter staging before its own Pop(b) issues. The consumer's serial issue discipline prevents that demand until Pop(a) retires. Hence at most one unmatched item is outside ingress: with F=1 the lazy inventory is at most two unmatched items, even though physical capacity is three. A tighter outstanding-credit cap can reduce this. Full F+m utilization requires a separately qualified forwarding/resource profile, such as elastic staging.

All demand decisions come from the pre-HLS run. No RTL request signals are consulted.

References: K.CT.LazySimulation.Example, LazyInventory, LazySlack.

Rules: K.CT.LazySerial · K.CT.LazySimulation

Why source demand supports RTL completion



Ordinary Push cases keep the standard floor proof. Demand support replaces only qualified extra-slack releases; no completion is fabricated for a pending demand.

The proof derives RTL progress from source evidence. The simulator does not observe it.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

A source demand record is evidence for a progress proof, not a claim that an RTL request has already occurred. Its prospectively fixed support includes operation/site identity, owner or complete batch, ordinal, matching admission, lower-ordinal retirements, and complete preparation/CtrlSupport. Alignment plus DemandPlacement or BatchPlacement derives the corresponding RTL request from completed support.

For an extra-slack Push(k), A=k-r=F and credit must exceed F (or be absent). All lower Pops are complete. If Pop(r) has retired, ingress has space. If item r is already outside ingress, it already released a slot. Otherwise placement gives the isolated pending request and qualified independent forwarding moves the oldest item out. Endpoint order and no stealing make that space persistent until the target Push. Ordinary Push cases retain the standard floor proof. The producer's own placement, guards and other resource conjuncts remain obligations.

Atomic-pair gain: let c have F_c=1 plus independent abstract staging and credit C_c>1 or no credit cap. The producer executes c.Push(a), c.Push(b), e.Push(x). The consumer issues the qualified same-site atomic input batch {Pop_c, Pop_e}. Channel e is an eligible independent staged path or an unreserved positive FIFO with space for x. The whole batch must satisfy BatchPlacement, with arrival-independent assertion and independent member forwarding. Pop_c's issued demand lets a leave c's ingress, admits b, and allows the producer to reach e.Push(x). The group can then retire. The floor-only graph cycles from group completion to c.Push(b), then e.Push(x), then group completion. Member-demand support avoids requiring group completion merely to release c's ingress. Under the standing qualification and service premises, RTL completes too.

A pending singleton can instead be completed in a protected finite source extension when its item is admitted, lower Pops retired and preparation complete. Clocks/guard opportunities continue, the testbench permits continuation, and no affecting reset, cancellation or termination removes needed work until the fixed finite demand set retires. Group completion is not obtained by this extension when a sibling admission is missing. Facts must cover newly consumed completions; Agreement covers the actual extended witness. RTL service is still required only until R.

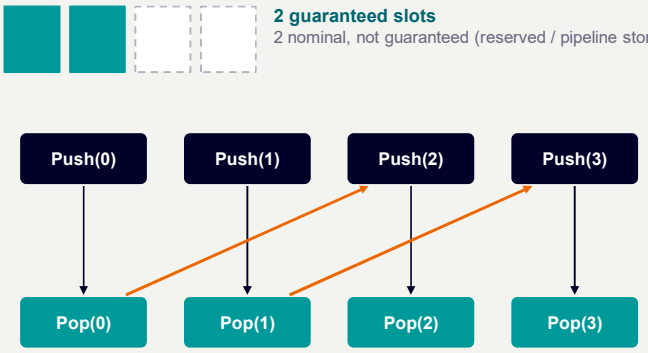
Without extension, auxiliary demand nodes supply dependencies but no fabricated completion facts. Their elimination leaves a graph on actual completed witness units. Facts applies over every admitted RTL finite prefix. Agreement and the support inventory cover every fact the certificate consumes.

References: K.CT.RecordedDemand, DemandPlacement, BatchPlacement, DemandPushResource, LazySourceExtension, LazyMixedRank, LazyComposition; guide §8.

Rules: B.CT.FIFO · K.CT · Guide §8

The standard floor route remains useful

Nominal capacity 4, guaranteed floor g = 2



Standard floor edges: Pop(k − g) precedes Push(k) for k ≥ g, even if the witness used all four slots.

Standard space proof

Standard FIFO-space dependencies use the guaranteed floor g, not nominal capacity.

Pipeline storage is qualified

Reserved or staged storage does not automatically enlarge the guaranteed floor.

Eligible demand alternative

Extra-slack cases can use demand support. Ordinary Push cases retain the floor proof.

Unbounded is not evidence

An unbounded run does not establish finite-hardware completion. Failure is inconclusive.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

This slide describes the standard floor resource proof, which remains useful alongside lazy completion. Nominal modeled storage does not automatically provide the standard guaranteed floor g. The standard certificate requires Pop(k-g) before Push(k) for k >= g. In the illustrated capacity-four/floor-two example, these are Pop(0) before Push(2) and Pop(1) before Push(3). Preserve the boundary case k=g.

For the concrete obstruction, explicitly take two ingress entries plus two optional abstract staging locations, with no tighter credit restriction. The producer sends three data items and then a start message. The consumer waits for start before issuing any process-facing data Pop. A permissive source run may move items early and accept all three sends. The floor-two certificate has a cycle: the first Pop needs start, start follows the third Push, and that Push needs the first Pop under the standard floor proof.

Lazy mode does not resolve this example. No data Pop has issued, so no data item may leave ingress under lazy selection. The third Push stalls in the pre-HLS run. A controller permitted to hold these items could deadlock in RTL. The lazy failure remains inconclusive about a particular RTL implementation. Certifying this case requires a stronger justified forwarding/resource guarantee, such as a qualified elastic-staging profile.

A separately justified smaller-capacity witness can also expose this obstruction, but requires its legal K.CT.Capacity lift. Lazy mode keeps the original elaborated model and capacity and requires no such capacity lift merely to impose its optional-movement restriction.

Retain the standard route for ordinary Push cases, eligible witnesses that use it, components outside lazy eligibility, and protected singleton extensions that supply actual floor predecessors. Demand support replaces only qualified extra-slack releases, never arbitrary missing dependencies. An unbounded-channel pass is not finite-hardware completion evidence.

References: B.CT.FIFO; K.CT.Standard, Capacity, LazySlack, DemandPushResource; guide §8.

Illustrative example · Rules: E4, J and K.CT

Worked example: four values through a finite FIFO

Configured system

One producer sends 10, 20, 30, 40 through a two-slot FIFO to a testbench receiver.

The model

- Producer: four ordered blocking Push calls
- Testbench: four ordered blocking Pop calls
- One clock, initially empty FIFO, B = g = 2
- Fixed values; no timing-dependent branches

No other producers, hidden storage or affecting resets

The claims

The required workload is the testbench's acceptance of all four values in that order.

Two separate obligations

- Safety: accepted values form a prefix of the list
- Completion: all four acceptances eventually occur
- The source safety property is proved separately
- Progress needs controller and service evidence

The source and RTL may use different cycle schedules

Ordinary unreserved FIFO: no lazy movement restriction. A finite accepting run supplies the witness.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Use one producer and one testbench receiver connected by an initially empty ordinary FIFO of capacity two. The producer makes four blocking Push calls with values 10, 20, 30, 40; the receiver makes four ordered blocking Pop calls. Both obey the registered-return and endpoint-order rules. There are no other producers, cancellations, affecting resets or timing-dependent branches. Both slots are usable, so B = g = 2.

The required workload is the four receiver acceptances, with those values in that order. The source safety property is proved separately: every accepted sequence is a prefix of the list, with no extra or duplicate values. The safety correspondence then carries that property to qualified RTL. A stalled run can satisfy the property without accepting all four values, which is why completion needs a separate argument.

A legal finite accepting source run supplies the completion witness. Here it can alternate Push(0), Pop(0), Push(1), Pop(1), Push(2), Pop(2), Push(3), Pop(3), using successively later legal cycles. This is deliberately conservative; independent processes need not generally be serialized together. The run uses neither fall-through nor replacement of an item on the same edge that a full FIFO removes one.

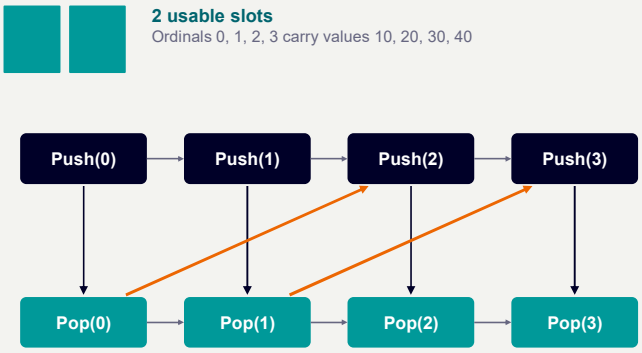
One such run supplies the witness only. It does not by itself prove the universal source safety property, source/RTL agreement or controller progress. The next slide shows the prerequisite graph and the additional progress conditions.

(References: Core.PolicyS, E4, J.SafeTransfer-HistCov and K.CT.Witness/Closure/Completion.)

This is an ordinary unreserved FIFO, with no attributed slots or optional staging movement to restrict. Within the shared restricted profile, the lazy resource construction reduces to the ordinary floor argument here. The existing example and graph therefore remain applicable without a new simulator mode. (K.CT.LazySerialScope; guide §8.)

Worked example: space, order and eventual acceptance

Ordinary unreserved FIFO: B = g = 2



Space prerequisites: Pop(0) before Push(2), Pop(1) before Push(3). Per-endpoint order also applies.

A legal finite witness

Push(0), Pop(0), Push(1), Pop(1), … on successive legal cycles accepts all four values.

No dependency cycle

Each item needs its Push. Reusing a slot needs an earlier Pop. Horizontal arrows preserve endpoint order.

What makes it progress

Finite preparation, persistent requests, continued clocks and receiver service expose each next transfer.

The conclusion

With source/RTL correspondence and progress proved, all four values are accepted in order. No cycle bound follows.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The graph includes three kinds of prerequisite. Each Push must precede its matching Pop. Each endpoint keeps its own order, shown by the horizontal arrows. With two usable slots, Pop(0) must precede Push(2), and Pop(1) must precede Push(3), shown in orange.

The alternating witness respects every arrow. Its eight transfers supply the required supporting work, and its four Pops are the receiving events we require (the formal sets W and R). Values and decisions are fixed and do not depend on timing. This example therefore has no cycle in its prerequisite graph.

Actual RTL may use a different schedule. Once the predecessors are complete, the controller must finish the necessary local preparation and hold the next request. For this ordinary FIFO, the item or free slot must exist at the start of the cycle. A full FIFO cannot use a same-edge Pop to justify a Push, and an empty FIFO cannot use a same-edge Push to satisfy a Pop.

Continued applicable clock edges then force transfers whenever settled ready and valid are both high. There is no arbiter in this example and no separate selection decision after a handshake becomes enabled. Required preparation and receiver service still need evidence that they occur.

With the source/RTL correspondence, agreement and progress conditions established, follow the prerequisites through the finite graph: every required Pop eventually accepts its expected value. This gives no fixed cycle count or throughput guarantee.

(References: B.CT.FIFO, Core.TransferLaw, K.CT.Closure, Exposure, Service and Completion; guide §8.)

Rules: K.CT.LazyComposition / Standard · Guide §§8–9

Completion scope and additional qualification

Restricted lazy profile

Eligible independent staging plus ordinary unreserved FIFOs.

Needs

- Serial or group-serial issue by every owner
- Lazy movement on optional abstract staging
- The prescribed complete certificate
- Rank lemmas then derive acyclicity

`K.CT.LazySerialRestricted`

Mixed composition

Lazy support on selected eligible channels; other components keep their own proofs.

Needs

- Serial or group-serial consumer per selected channel
- Arrival-independent request assertion
- Ungated staging; whole-batch eligibility
- Combined graph proved or checked acyclic

`K.CT.LazyComposition`

Standard or extended routes

Floor proofs and separate qualifications remain available.

Need additional proof

- General arbiters or shared credit pools
- Timing-dependent polling or signal control
- Staging outside the named delivery profiles
- Reset crossing outstanding required work

`K.CT.Standard · Q.CT.Extension`

Separate obligations: acyclicity, complete dependencies and valid exposure proofs. A failed sufficient route is inconclusive.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

There are three complementary applications. The restricted lazy profile uses initially empty exclusive independent InputStaging paths plus ordinary unreserved FIFOs, the prescribed complete certificate, and serial or qualified group-serial issue by every relevant owner. Its rank lemmas derive acyclicity. Mixed composition instead applies demand support locally and establishes acyclicity of the assembled graph. The standard route and separately proved extensions remain available.

Selected lazy channels require ordered exclusive storage, declared ingress and credit laws, optional abstract movement, and no additional producer epoch gate or gated internal movement. Their consumers use serial/group-serial issue and satisfy DemandPlacement or BatchPlacement, including source-control conformance, complete preparation/CtrlSupport and arrival-independent request assertion. Data-driven pre-request placement does not inherit these lemmas. Producer serial issue is unnecessary for the local resource lemma, but producer endpoint order, source legality and exposure remain obligations.

For an atomic member, the whole batch must qualify: pure positive, same site, independently prepared, with each positive channel occurring at most once. Members may be independent InputStaging or ordinary unreserved FIFOs with coincident capture and retirement. Attributed-slot split capture, a coupled or staggered member, or a rendezvous leg does not qualify for this batch proof.

Rendezvous, synchronization, gates, coupled/staggered storage and reserved-slot components may exist elsewhere in a mixed design. Retain their own closure, resource, exposure and service proofs, preserve atomic units, and include all cross-component dependencies. A topological sort proves only acyclicity, not inventory completeness or exposure validity.

Select channels independently of the RTL execution being proved. Selection after source simulation is permitted only when the recorded trace, including an extension, actually obeys the lazy restriction. It cannot relabel eager moves as lazy.

Synchronization does not reset global item ordinals. Under the applicable serial consumer discipline, a pending demand does not cross a sync boundary. This does not waive gate restoration or any other resource conjunct.

Timing-dependent control, general arbitration/shared credit, unsupported delivery profiles and resets crossing required work retain their qualifications. Failure of a sufficient condition is inconclusive.

References: K.CT.LazyEligibility, LazyComposition, LazySerialRestricted, Standard; B.CT.Compatibility; Q.CT.Extension; guide §§8–9.



It is just as important to know what is not claimed.

No reverse realization: the RTL need not reproduce an arbitrarily chosen source simulation. Appendix L can align a source run to a selected RTL run, but that is a verification technique, not a theorem that every source behavior is realizable.

No cycle equality: identical timing, registers, latency or throughput are not implied; they need their own timing model and evidence.

No cycle bound: completion is 'eventually'.

No total deadlock freedom: deadlocks in unrelated work or after the required outputs are accepted are outside the result.

No automatic tool certification: the proofs do not certify a particular HLS invocation or RTL instance; the flow must supply evidence. The same framework can apply to hand-written RTL if equivalent evidence is supplied.

Scope limits: one global clock per theorem instance - Appendix O only sketches multiclock - and asynchronous reset assertion needs a separate argument.

And when a sufficient check fails, the result is inconclusive; it does not prove the design wrong.

Dynamic reset may qualify for safety under its own profile. For completion, an affecting reset, cancellation or premature testbench termination cannot remove outstanding required work without a separate preservation qualification. Required clocks and environment service continue until acceptance.

A failed lazy witness exposes an obstruction in that conservative source execution; it does not prove an RTL deadlock. A qualified passing lazy witness establishes the stated workload result, not latency, throughput or a universal source safety property. (K.CT.LazySerialScope, LazyComposition.)

Design constraints for architects

1 Anchor sequential signal IO
Preserve whether each observable read occurs, its value and its sync point.

2 Stable direct inputs
Stable per reset epoch, or updated only via the sync handshake.

3 Explicit timing sensitivity
Latency-dependent environment behavior isolated or made observable.

4 Polling is a design decision
Non-blocking control picks a proof route; prefer blocking calls.

5 No unbounded busy polling
Qualify retries under the chosen profile, or use a blocking call.

6 Expose shared-memory values
Each cross-process memory dependency via a supported boundary.

7 Model the real channel
No hidden fall-through, bypass, debug drain or side entrance.

8 Elaborate joined structures
Joins, coupled multi-input pipelines and sync gates modeled explicitly.

9 Reset with a disposition
Reset both channel ends together, or define a flush / abort protocol.

For completion: prefer lazy mode where eligible; qualify resources and joint transfers, use real capacities and maintain service. Required work needs reset/cancellation protection.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

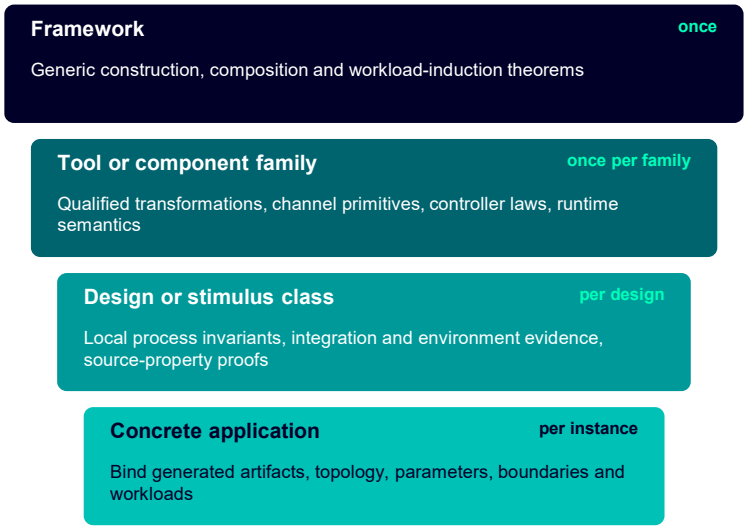
Appendix F gathers the design choices that most directly decide whether the guarantees can be applied. A design can synthesize and work correctly and still sit outside the theorem scope if one of these is left implicit.

Briefly: give every signal access an unambiguous anchor. Keep direct inputs stable or update them only through the sync handshake. Make timing-sensitive environment behavior explicit. Treat non-blocking polling as an architectural decision about which proof route you are taking, and do not rely on unbounded busy polling. Expose cross-process shared-memory dependencies. Model the channel behavior the RTL actually provides - no hidden fall-through, bypass, debug drains or side entrances. Use an elaborated model for joins, coupled multi-input pipelines and sync gates. And give every dynamic reset a channel-consistent disposition - reset both ends together, or define an explicit flush or abort protocol.

For completion, add: point-to-point channels with explicit arbiter processes, explicit multi-lane transfers, persistent blocking requests, real capacities, and explicit barrier, end-of-stream and abort protocols.

For completion, prefer lazy mode locally where eligibility holds. The simulator must expose the process-facing Pop demand once legally reached and prepared, independently of item arrival. Declare which staging moves are abstract and which boundaries are physical ready/valid interfaces. The restriction cannot suppress a mandatory physical transfer. Atomic batches must qualify as whole units. (K.CT.LazyEligibility, LazySimulation.)

Who supplies which evidence



The division of labor

- Architects choose interfaces, reset and communication structures that admit the correspondence.
- The tool and verification flow must supply or check the detailed evidence for the claimed scope.
- Evidence: documentation, lint and static checks, targeted RTL / interface checks, formal verification.
- Written with the precision intended to support mechanization in Lean.

Most of these conditions were always there. The formalization names them, so they can be audited instead of quietly assumed.

Lazy evidence comes from the pre-HLS run; no extra RTL run per stimulus.

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The formal appendices are large, and it is fair to ask who has to deal with all of that.

The answer is layered. The framework - the generic construction, composition and workload-induction theorems - is proved once. A tool or component family, such as Catapult and the Matchlib primitives, is qualified once per family: transformations, channel primitives, controller laws, runtime semantics. A design or stimulus class supplies its local process invariants, integration and environment evidence, and the independent source-property proofs. A concrete application binds the generated artifacts, topology, parameters and workloads.

The architect's job is to choose interfaces, reset behavior and communication structures that admit the correspondence. The tool and verification flow construct and check the detailed records. Evidence can come from documentation, static checks and lint, targeted RTL or interface checks, or formal verification - and the appendices are precise enough to support mechanization in Lean.

Most of the engineering conditions were always there. The formalization just names them, so they can be audited instead of quietly assumed.

The evidence division is a responsibility model, not a claim that every record is currently generated automatically. Reusable qualifications bind the relevant tool/runtime versions, libraries, feature profiles and parameters. Changes outside that domain require renewed evidence.

Lazy evidence adds reusable component/consumer progress and simulator-semantics qualification, exact instance binding, and workload records from the pre-HLS trace. M.7a records selected-channel eligibility, lazy restriction checks, capacity/credit and boundary classification, complete dependency/control support, demand or extension records, and restricted rank evidence or assembled-graph acyclicity. Extension records include the protected whole-design continuation window. Lazy simulation itself requires no RTL request, progress or completion observation per stimulus. Existing structural/implementation checks and any separately selected snooping remain distinct. This responsibility model does not claim that all records are already produced automatically.



The scheduling contract lets most functional verification and debugging happen at the source while HLS retains freedom to optimize. Its guarantees depend on the declared models, interfaces, implementation behavior and supporting evidence.

A source interface property proved for all admitted source behaviors also holds on covered RTL behaviors once the safety correspondence is established. Required-workload completion additionally needs a legal finite accepting source run, agreement on internal values and decisions, a complete prerequisite graph, controller and resource progress, and continuing service.

Architects choose communication, memory, reset and timing structures that support the intended claim. The tool and verification flow supply or check the detailed evidence for the supported configurations. The framework specifies those obligations; it does not automatically certify a particular tool flow.

The contract is tool-neutral and provides a starting point for standardization in the Accellera Synthesis Working Group.

(References: Formal Guarantees at a Glance, Appendices J, K and M; guide §§1 and 11.)

Prefer lazy pre-HLS completion checks where eligible; they add no per-stimulus RTL simulation or probe, but the reusable conformance, instance binding, Facts/Agreement, exposure and service evidence remains necessary. The standard floor route remains useful for ordinary cases and other qualified components.

REFERENCES AND FURTHER READING

References and further reading

Repository github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG

Catapult HLS User View Scheduling Rules (26 October 2026)
.../matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

Formal Architecture Guide (26 October 2026)
.../matchlib_examples/doc/formal_architecture_guide.pdf

Brief summary of the scheduling rules
.../matchlib_examples/doc/brief_scheduling_rules.txt

Examples cited: 09, 12, 15, 32, 53, 61
.../matchlib_examples/examples

Making shift-left real (white paper)
hls.academy/making-shift-left-real-why-systemc-matchlib-and-high-level-synthesis-...

Verifying hardware at the source (white paper)
hls.academy/topics/systemc-modeling/verifying-hardware-at-the-source-...

Reading paths in the rules

Architects and HLS users

Main body through App. F; App. B and C; App. K including LazySerial

Safety reviewers

Normative Formal Core; App. G; App. J constructive theorems

Overview and evidence

App. M formal results; M.7a evidence checklist

 Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Exact revision baselines for this deck:

catapult_user_view_scheduling_rules_26oct26_lazy_workflow_revised.docx

formal_architecture_guide_26oct26_lazy_aligned.docx

The repository URLs on the slide identify publication locations; their published PDFs may represent another revision. Do not assume they already contain these updates.

Architects and HLS users: main body through Appendix F, Appendices B/C, and K.CT.LazySimulation for the operational meaning and example. K.CT.LazyComposition is the general mixed-component theorem. K.CT.LazySerialRestricted supplies rank-derived acyclicity in its restricted profile. K.CT.LazyExtendedStandard is the separate protected-extension return to Standard.

Safety reviewers: Normative Formal Core, Appendix G and constructive Appendix J results. Appendix M describes formal results; M.7a lists instance and lazy evidence. The architecture guide §§8–11 explains completion, mixed scope and evidence responsibility.